



Optimization and Experimental Evaluation of the SOX Fair Exchange Implementation

Ryad Mohamed Aouak

School of Computer and Communication Sciences

Semester Project

Spring 2026

Responsible

Prof. Serge Vaudenay
EPFL / LASEC

Supervisor

Prof. Serge Vaudenay
EPFL / LASEC



Contents

1	Introduction	1
1.1	Context and motivation	1
1.2	Terminology used throughout the report	2
1.3	Project goals	2
1.4	Methodology	4
1.5	Main contributions	5
1.6	Scope and limitations of the project	7
1.7	Report structure	7
2	Background and Protocol Foundations	9
2.1	Fair exchange and optimistic protocols	9
2.2	Overview of the SOX protocol	10
2.3	Roles, sponsors, deposits, and incentives	11
2.4	Optimistic path of the protocol	12
2.5	Dispute path of the protocol	13
2.6	Circuit commitments: hct, hcircuit, and hpre	14
2.7	ERC-4337 and transaction sponsorship	15
2.8	Security assumptions and fairness goals	16
3	Initial Implementation Analysis	19
3.1	Repository structure and technology stack	19
3.2	Rust and WebAssembly off-chain pipeline	21
3.3	Frontend, backend, database, and file storage	22
3.4	Optimistic smart contract architecture	23
3.5	Dispute smart contract architecture	23
3.6	ERC-4337, Alto bundler, and EntryPoint integration	24
3.7	Initial end-to-end execution flow	25
3.8	Limitations of the initial implementation	26
4	Methodology and Experiments	29
4.1	Project methodology	29
4.2	Reproducibility strategy	30
4.3	Mapping code functions to protocol steps	31

4.4	Gas accounting scopes	32
4.5	Off-chain timing methodology	33
4.6	Benchmarking environment	34
4.7	Validation strategy	34
5	Protocol Variants and Design Choices	37
5.1	Motivation for protocol variants	37
5.2	Normal sponsored SOX	39
5.3	No S-deposit and simSOX	39
5.4	S=B: sponsor is buyer	40
5.5	S=V: sponsor is vendor	41
5.6	SB=B and SV=V: self-sponsored disputes	41
5.7	Hardcoded SHA-256 circuit variant	42
5.8	Variants kept, variants discarded, and rationale	43
6	Implementation Architecture	47
6.1	From monolithic contracts to specialized contracts	47
6.2	Frontend and backend mode selection	48
6.3	Factory and optimistic clones	49
6.4	Specialized optimistic escrow contracts	50
6.5	Specialized dispute game contracts	51
6.6	Hardcoded SHA-256 dispute contracts	51
6.7	Native CLI path for large files	52
6.8	Database and artifact management	53
6.9	Integration with the existing application	53
7	Correctness and Security Analysis	55
7.1	Functional correctness of the optimistic path	55
7.2	Functional correctness of the dispute path	56
7.3	Correctness of the hardcoded SHA-256 mode	57
7.4	Sponsor safety and incentive preservation	58
7.5	Encoding ambiguities in the inherited circuit format	58
7.6	Merkle tree domain separation issues	59
7.7	ERC-4337 trust and liveness assumptions	60
7.8	Remaining security limitations	60
8	Experimental Results	63
8.1	Measurement methodology and scopes	63
8.2	Baseline measurements	65
8.3	One-time infrastructure deployment costs	65
8.4	Optimistic path gas costs	66
8.5	Dispute path gas costs	67
8.6	Hardcoded SHA-256 gas impact	68
8.7	Self-sponsoring gas impact	69

8.8	Large-file measurements up to 1 GiB	69
8.9	Off-chain runtime measurements	70
8.10	Comparison summary	72
8.11	Interpretation of results	72
9	Discussion	75
9.1	What actually reduced gas	75
9.2	What did not reduce gas as expected	76
9.3	Cost relocation versus true optimization	77
9.4	Trade-offs of specialization	78
9.5	Practicality of large-file fair exchange	78
9.6	Lessons learned from the implementation	79
10	Limitations and Future Work	81
10.1	Current prototype limitations	81
10.2	Verification workflow limitations	82
10.3	Browser and WASM limits	83
10.4	Registry-first architecture	83
10.5	Step 1+2 fusion for S not equal to B	84
10.6	Cleaner circuit encoding	85
10.7	Merkle domain separation	85
10.8	Wallet integration	86
10.9	Production deployment considerations	87
11	Installation and Demonstration	89
11.1	System requirements	89
11.2	Repository setup	90
11.3	Building the Rust and WebAssembly components	90
11.4	Deploying the local smart contract stack	91
11.5	Running Hardhat, Alto, and the frontend	92
11.6	Running the optimistic demo	92
11.7	Running gas benchmarks	93
11.8	Running large-file benchmarks	94
11.9	Troubleshooting	94
12	Conclusion	97
12.1	Summary of the project	97
12.2	Summary of results	98
12.3	Final assessment	98
12.4	Perspectives	99

A	Detailed Gas Tables	101
A.1	Initial and monolithic optimistic paths	101
A.2	Factory and Clone Costs	101
A.3	Same-scope dispute comparisons	102
A.4	Hardcoded strict end-to-end path	103
A.5	Monolithic hardcoded mirror	103
B	Detailed Off-Chain Timing Tables	105
B.1	Native 1 GiB benchmark timings	105
B.2	First hpre rounds	106
B.3	Native versus browser/WASM interpretation	106
C	Protocol-Step Mapping Table	107
C.1	Scope definitions	107
D	Contract and Script Reference	111
D.1	Solidity contracts	111
D.2	Rust and application components	112
D.3	Scripts and benchmark files	113
E	Demo Workflow	115
E.1	Preparation	115
E.2	Optimistic exchange demonstration	115
E.3	Dispute demonstration note	116
E.4	Recording notes	116
F	Additional Reproducibility Notes	119
F.1	Clean measurement discipline	119
F.2	Canonical benchmark commands	119
F.3	Interpreting local benchmark output	120
F.4	Known local-environment issues	120
F.5	Submitted-code boundary	121

Chapter 1

Introduction

1.1 Context and motivation

Fair exchange is the problem of allowing two parties to exchange valuable assets in such a way that no honest participant is left at a disadvantage. In the setting considered in this project, a vendor V owns a digital item x , a buyer B wants to buy it for a price y , and both parties know in advance a public description predicate `desc` and an expected digest d . The intended outcome is simple to state: either B obtains an item x' such that `desc`(x') = d and V receives the payment, or the exchange is cancelled without giving either side an unfair advantage.

This problem is classical in cryptography, but it remains difficult to deploy in practice. Without an external trusted party, perfect two-party fair exchange is impossible in the general case. Smart contracts provide a natural alternative: the blockchain can act as a public, deterministic, and economically enforceable adjudicator. However, using a blockchain introduces another difficulty. Every on-chain action costs gas, and these costs can make the exchange unattractive, especially when the exchanged item is small, when one participant has no cryptocurrency available, or when a dispute forces expensive verification on-chain.

The Sponsored Fair Exchange protocol, denoted SOX in this report, addresses this tension by combining three ideas. First, it follows an optimistic design: when both parties behave honestly, the exchange should finish with very little on-chain computation. Second, it keeps a dispute mechanism able to resolve disagreements by verifying only a logarithmic number of positions in a computation trace. Third, it introduces sponsors, whose role is to cover gas costs under predefined incentives. The goal is not only fairness, but also economic accessibility: the vendor and the buyer should not need to manage gas in the common case.

The starting point of this semester project was an existing implementation of SOX, documented in the previous implementation report [2] and based on

the protocol paper [1]. That implementation already represented a substantial proof of concept. It contained a Rust/WebAssembly off-chain pipeline, Solidity contracts for the optimistic and dispute phases, an Electron/Next.js interface, and an ERC-4337-inspired transaction sponsorship path. It also introduced a compact block-based circuit representation for large files, with 64-byte gates and Merkle commitments over the circuit, ciphertext, and intermediate values.

The purpose of the present project is different from reimplementing SOX from scratch. The objective is to take this initial system seriously as a research prototype, analyze where its implementation diverges from the most gas-efficient protocol variants, and modify it so that the main variants suggested by the protocol analysis can be executed and measured. In other words, the project is both an implementation project and an experimental methodology project: the code must work, but the measurements must also correspond to well-defined protocol scopes.

1.2 Terminology used throughout the report

A recurring source of confusion in this project is that several implementations coexist in the repository and they do not measure the same thing. To avoid ambiguity, this report uses the following terminology consistently.

This distinction is essential for interpreting the experimental results. A number such as “optimistic cost”, “deployment cost”, or “dispute cost” is meaningful only after saying which implementation is measured and which protocol steps are included. One of the outcomes of this project is therefore a stricter accounting discipline: costs are reported either as one-time infrastructure deployments, per-exchange optimistic costs, dispute initialization costs, or per-round dispute execution costs.

1.3 Project goals

The project was initially motivated by the need to understand the existing SOX implementation and to make it executable on a complete local stack. After this first phase, the scope was refined around gas reduction and protocol variants, following discussions with the supervisor. The final goals can be summarized as follows.

1. Reconstruct and validate the execution path of the inherited implementation, from off-chain precontract generation to optimistic completion and dispute execution.
2. Identify which sponsoring variants actually affect gas costs. In particular, the project studies no S -deposit, $S = B$, $S = V$, $S_B = B$, and $S_V = V$,

Term	Meaning in this report
Initial implementation	The implementation inherited from the previous semester project. It is the baseline used to understand the existing architecture and to reproduce the original costs.
Monolithic implementation	The intermediate implementation in which several variants are supported by the same large contracts. This version is useful for validation but expensive because variant selection logic remains on-chain.
Specialized final implementation	The optimized implementation produced in this project, where the main protocol modes are split into smaller dedicated contracts or clone-based variants. The goal is to move mode selection off-chain and avoid paying gas for unused branches.
Hardcoded SHA-256 mode	A specialized dispute mode for the case $\text{desc} = \text{SHA256}$, where the circuit is determined by the input length and no generic circuit commitment proof π_1 is required in Step 8.
SIMSOX / no S -deposit	A simplified optimistic mode in which there is no initial sponsor deposit by S . Each participant pays gas in the optimistic part, removing the need for sponsor reimbursement logic in that part of the contract.

Table 1.1: Implementation terminology used in the report.

while separating variants determined at precontract time from variants determined only once a dispute starts.

3. Implement the main variants in Solidity without changing the intended protocol semantics. The implementation must avoid a “Swiss-army-knife” contract where all modes are present in one large bytecode and all users pay for unused logic.
4. Implement and measure a hardcoded **SHA256** dispute mode. For this mode, the circuit is fixed by $|x|$, so the smart contract can reconstruct the relevant gate in Step 8 instead of requiring a generic circuit gate and the proof π_1 .
5. Produce reproducible gas measurements for the optimistic path and the dispute path, including a clear distinction between one-time deployment costs and marginal per-exchange costs.
6. Test large-file execution, especially 1 GiB inputs, using the native Rust command line pipeline when the browser/WebAssembly execution path becomes impractical.

These goals are deliberately implementation-oriented. The project does not claim a new cryptographic proof of the SOX protocol. Instead, it asks whether the theoretical variants that are attractive on paper can be translated into contracts that are measurably cheaper and still faithful to the intended protocol flow.

1.4 Methodology

The work followed an iterative methodology. The first step was to make the inherited system run reliably in a local development environment. This required rebuilding the Rust and WebAssembly components, deploying the Solidity stack, restoring the expected local database and backend folders, and understanding the actual path used by the frontend. This phase was necessary because several important costs are not visible by reading a contract alone: the application decides when a contract is deployed, which role signs which operation, whether a transaction goes through ERC-4337 tooling, and which off-chain artifacts are stored or transferred.

The second step was protocol mapping. The implementation uses function names and UI actions that do not directly match the step names in the paper. For example, the dispute trigger, challenge response, opinion submission, commitment submission, and finalization functions correspond to different substeps of the paper protocol. Before measuring gas, these mappings had to be fixed explicitly. Otherwise, two measurements with the same informal label could include different pieces of the protocol.

The third step was variant isolation. The initial engineering temptation was to add feature flags to the existing contracts. This approach is convenient for development because one contract can simulate many cases. It is not, however, an adequate gas optimization strategy: Ethereum users pay for bytecode size, storage layout, and runtime branches even when a given exchange uses only one mode. The project therefore moved progressively from a monolithic implementation to specialized contracts, clone-based optimistic escrows, and dedicated dispute games.

The fourth step was measurement. Gas was measured through Hardhat tests designed to execute complete scenarios rather than isolated functions whenever possible. This matters because deployment and initialization can dominate the total cost. For large files, off-chain timings were measured with the native Rust command line tools instead of the browser path, because the browser/WebAssembly execution path is not the right environment for 1 GiB experiments. The measurements in the final report distinguish:

- one-time infrastructure deployment costs, such as deploying factories and implementations;
- marginal optimistic costs per exchange, including deployment or clone creation and execution until completion;
- dispute initialization costs, especially the cost of deploying or creating the dispute contract;
- post-trigger dispute execution costs, such as challenge rounds and Step 8 verification;
- off-chain timings for precontract generation and dispute witness computation.

The final step was consistency checking. Several early measurements were technically correct but difficult to interpret because they mixed scopes: some came from the monolithic version, some from the specialized version, some included deployment, and some did not. The report therefore treats measurement clarity as part of the contribution. A gas table is only useful if the reader can reconstruct exactly what was paid, by whom, and at which stage of the protocol.

1.5 Main contributions

The main contributions of this project are the following.

Variant-aware optimistic contracts. The optimistic phase was split into dedicated modes for the normal case, no S -deposit, $S = B$, and $S = V$. In the final specialized implementation, the mode is selected before deployment by the frontend/backend pipeline, so the contract does not need to carry all branches at runtime. This is the main reason why the marginal optimistic cost can be reduced substantially compared with the initial account-per-exchange design.

Self-sponsored dispute execution. The dispute contracts were specialized for cases where the dispute sponsors coincide with the protocol participants, in particular $S_B = B$ and $S_V = V$. These cases matter because they remove unnecessary repetitions of the Step 7–8 loop caused by Step 9 sponsor rotation. The project therefore separates the cost of one iteration from the cost of two or three iterations, rather than hiding this effect behind a single aggregate dispute number.

Hardcoded SHA256 dispute mode. For the important case $\text{desc} = \text{SHA256}$, the circuit is determined by the input length. A dedicated hardcoded dispute contract was implemented so that Step 8 no longer requires the generic circuit proof π_1 . In the strict specialized mode, the contract reconstructs the relevant SHA-256 gate internally; the vendor does not provide the generic gate bytes used by the monolithic verifier. This distinction is important because the monolithic hardcoded variant can remove π_1 but still requires a 64-byte gate encoding, whereas the specialized hardcoded variant removes both.

Large-file execution path. The project validated an execution path for large files up to 1 GiB using native command line tooling. This does not mean that every browser action is suitable for 1 GiB inputs. Rather, it establishes a practical split: the UI remains useful for demonstration and small end-to-end exchanges, while the native Rust pipeline is the correct measurement path for large-file precontract generation and dispute witness computation.

Reproducible gas accounting. The project produced benchmark tests for the initial, monolithic, and specialized versions. It also clarified which costs are one-time protocol deployment costs and which costs are paid per exchange. This distinction is central to evaluating whether a change is a true gas optimization or only a relocation of cost from one part of the system to another.

1.6 Scope and limitations of the project

The project focuses on implementation, measurement, and architecture. It does not attempt to replace the security proof of the SOX protocol. It also does not claim that the current code is production-ready. The implementation remains a research prototype, and several limitations remain important.

First, the current application still uses a laboratory identity model with local accounts and development-chain deployment scripts. Wallet integration would be necessary for a realistic user-facing deployment, but it was not the main objective of this project.

Second, large-file execution is validated through the native command line pipeline, not through a fully browser-based flow. This is intentional: the original browser/WebAssembly path is convenient for small demonstrations, but it is not the right tool for measuring 1 GiB precontract and dispute computations.

Third, the project revealed questions about the inherited circuit encoding and Merkle tree domain separation. These questions concern the historical circuit format and are not caused by the gas-specialization work itself, but they are relevant for any future production-grade version. A clean final design should make gate encodings unambiguous and should separate Merkle leaves from internal nodes by domain tags.

Finally, some architectural improvements remain future work. In particular, a more radical factory or registry architecture could avoid deploying a full optimistic account for every exchange. The present project already moves in that direction with specialized contracts and clones, but a fully reusable registry-style deployment model would require a broader redesign of the inherited application.

1.7 Report structure

This first chapter defines the problem, the terminology, the goals, and the methodology of the project. The remaining chapters will describe the inherited implementation, the protocol variants, the specialized contract architecture, the experimental measurements, the security and encoding issues identified during the project, and the installation and demonstration workflow. The rest of the report will keep the terminology introduced in Table 1.1 so that gas costs can be compared without mixing the initial, monolithic, and specialized implementations.

Chapter 2

Background and Protocol Foundations

This chapter gives the protocol background needed to understand the implementation work presented later in the report. It does not introduce new measurements and it does not claim a new proof of security. Its purpose is to fix the vocabulary, the protocol objects, the role of the sponsors, and the distinction between the optimistic execution path and the dispute execution path. This is important because most of the engineering choices made in this project are only meaningful once the scope of each protocol step is unambiguous.

2.1 Fair exchange and optimistic protocols

A fair exchange protocol allows two parties to exchange valuable objects without trusting each other. In the setting studied in this project, the vendor V owns a digital item x , and the buyer B is willing to pay a price y if the item satisfies a public description predicate. Informally, the desired outcome is:

$$B \text{ obtains a valid item } x \iff V \text{ obtains the payment } y.$$

If the exchange cannot complete, neither side should obtain a strictly better position by deviating from the protocol. A dishonest vendor should not be able to get paid for an invalid or unavailable item, and a dishonest buyer should not be able to obtain the item while recovering the payment.

Perfect two-party fair exchange is known to be impossible in the general asynchronous setting without an additional enforcement mechanism. Blockchain-based protocols use the smart contract as such a mechanism. The contract is not a trusted party in the traditional sense: its code is public, deterministic, and executed by the blockchain. It can nevertheless hold funds, enforce deadlines, verify cryptographic evidence, and redistribute deposits when one party refuses to continue.

The difficulty is cost. A naive smart-contract solution would put the item, the verification logic, or a large part of the computation on-chain. This is not viable for large files. The gas cost of storing data, deploying bytecode, and executing cryptographic computations grows quickly and can dominate the value of the exchange. The protocol therefore follows an *optimistic* design: the common case should be cheap, and expensive verification should only happen if a dispute is raised.

An optimistic fair exchange protocol separates two paths:

- the *optimistic path*, where both parties behave as expected and the exchange completes after a small number of transactions;
- the *dispute path*, where one party claims that the exchange is invalid and the contract adjudicates the claim using a compact proof.

This separation is central to SOX. The protocol accepts that a dispute may be expensive, but it tries to make disputes rare, logarithmic in the size of the computation trace, and economically financed by sponsors and deposits.

2.2 Overview of the SOX protocol

The Sponsored Fair Exchange protocol, abbreviated SOX, is described in [1]. The implementation inherited at the beginning of this project follows the same high-level architecture and was documented in the previous implementation report [2].

The digital item is not sent directly to the buyer in clear. Instead, the vendor encrypts the item and publishes an encrypted object ct . The buyer can download ct , but cannot recover x until the vendor reveals the decryption key. Before the payment is released, the vendor also commits to the data and to the computation that should prove that the plaintext satisfies the expected description.

The protocol is built around the following public and authenticated objects:

- the ciphertext ct , which is the encrypted representation of the item;
- the description $desc$, for example SHA256 in the main specialized case studied in this project;
- the expected digest or description output d ;
- a circuit C that evaluates whether decrypting ct gives an item satisfying $desc(x) = d$;

- cryptographic commitments to ct , to the circuit, and to intermediate values used during dispute resolution.

The smart contract does not store the whole file and does not execute the full circuit in the optimistic case. It only stores short commitments and enough economic state to decide who should receive the funds. If the buyer accepts the decrypted item, the contract releases the payment to the vendor. If the buyer complains, the dispute mechanism narrows the disagreement to a single circuit position and verifies a local computation step on-chain.

The important architectural idea is that the contract only needs to verify a small local claim after a logarithmic search. The expensive work of encrypting, constructing the circuit, computing intermediate values, and preparing Merkle proofs remains off-chain. The on-chain part checks enough authenticated data to make lying economically and cryptographically detectable.

2.3 Roles, sponsors, deposits, and incentives

The protocol distinguishes between the parties who exchange the item and the parties who pay gas or finance parts of the protocol. In the most general form, there are five roles:

Role	Function in the protocol
V	Vendor. Owns the item x , prepares the encrypted item and the precontract data, reveals the decryption key, and defends the correctness of the exchange during a dispute.
B	Buyer. Accepts the precontract, pays the price, decrypts the ciphertext after receiving the key, and raises a complaint if the decrypted item does not satisfy the description.
S	Optimistic sponsor. In the sponsored version, this role finances or deposits funds for the optimistic part of the protocol.
S_B	Buyer-side dispute sponsor. This role finances the buyer side of the dispute when the buyer claims that the exchange is invalid.
S_V	Vendor-side dispute sponsor. This role finances the vendor side of the dispute when the vendor defends the validity of the exchange.

Table 2.1: Protocol roles used by SOX.

The motivation for introducing sponsors is practical. A buyer or vendor may not want to hold gas funds, or may not want to expose themselves

to the full execution cost of a dispute. Sponsors can cover these costs in exchange for deposits, tips, or reimbursement rules encoded in the contract. The exact economic values are implementation parameters, but the logic is protocol-driven: if a party causes an unnecessary or losing dispute, the deposits should compensate the party who paid for the successful execution.

Several roles may coincide. For example, $S = B$ means that the buyer is also the optimistic sponsor, $S = V$ means that the vendor is also the optimistic sponsor, $S_B = B$ means that the buyer sponsors its own side of the dispute, and $S_V = V$ means that the vendor sponsors its own side. These equalities are not cosmetic. They can remove transactions, deposits, or repeated dispute iterations. A large part of this project studies which coincidences have a real gas impact and which ones are mostly semantic.

One variant is especially important: the no S -deposit mode, also called SIMSOX in the measurements. In that mode there is no initial sponsor deposit by S in the optimistic part. The parties pay for their own optimistic transactions, so the contract no longer needs to carry reimbursement logic for an external optimistic sponsor. This simplifies the optimistic contract and is one of the main reasons for the lower marginal cost measured in the final implementation.

2.4 Optimistic path of the protocol

The optimistic path is the path followed when the vendor provides a valid item and the buyer accepts it. In the inherited implementation and in the specialized variants developed during this project, the path can be summarized as follows.

1. **Precontract generation.** The vendor prepares the encrypted file, the public description data, and the commitments required by the protocol. This computation is off-chain and may be expensive for large files.
2. **Contract creation or initialization.** The optimistic contract for the exchange is deployed or created through the selected deployment mechanism. In the initial implementation this was essentially an account-per-exchange design. In the final specialized implementation, several modes use a factory and smaller dedicated contracts.
3. **Buyer payment.** The buyer sends the payment to the optimistic contract. The contract escrows the payment instead of transferring it immediately to the vendor.
4. **Key release.** The vendor reveals the symmetric key. The buyer can now decrypt the ciphertext and check the item locally.

5. **Completion or complaint.** If the buyer is satisfied, the buyer completes the exchange and the contract releases the payment to the vendor. If the buyer is not satisfied, the protocol leaves the optimistic path and enters the dispute path.

In the Solidity implementation, the optimistic contract states reflect this structure. The contract waits for the buyer payment, then waits for the vendor key, then waits for either buyer completion or dispute preparation. The exact state names vary slightly across the initial, monolithic, and specialized contracts, but the core sequence remains:

`WaitPayment → WaitKey → WaitSB → WaitSV or End.`

The state **End** corresponds to a completed or otherwise finalized exchange.

The gas objective for this path is strict: the optimistic path is the common case, so it must be as cheap as possible. Any branch that only matters in rare dispute scenarios should not be paid by every honest exchange if it can be moved to a specialized contract or delayed until a dispute actually occurs.

2.5 Dispute path of the protocol

A dispute starts when the buyer claims that the decrypted item is invalid. The purpose of the dispute is not to recompute the whole circuit on-chain. Instead, the parties conduct an interactive search over the computation trace until the disagreement is localized. Then the contract verifies one local computation step with Merkle proofs and the relevant gate evaluation.

At a high level, the dispute mechanism has three phases.

Dispute initialization. The buyer announces dissatisfaction and the dispute infrastructure is initialized. In the implementation, the operation usually called `triggerDispute` corresponds to the paper’s Step 6. It is important to note that in the measured implementation this action can include deployment or creation of the dispute contract. Therefore, its gas cost is not merely the cost of changing an optimistic state variable.

Binary search over the computation trace. The buyer and the vendor narrow the interval in which they disagree. The implementation uses `respondChallenge` for the buyer response and `giveOpinion` for the vendor’s opinion. These correspond respectively to Step 7b and Step 7c in the protocol mapping used throughout this report. The number of rounds is logarithmic in the number of gates. For example, if the circuit has N gates, the search requires roughly $\lceil \log_2 N \rceil$ rounds before it reaches a single local position.

Local verification and finalization. Once the relevant position is identified, the vendor submits the local data required to verify one gate or one boundary condition. In the implementation this is the `submitCommitment` family of calls, corresponding to Step 8. The contract checks membership proofs against the commitments, evaluates the local gate, and then finalizes or continues according to the protocol logic. The final transition is referred to as `finalize` in the benchmarks and corresponds to Step 9.

Implementation name	Paper step	Purpose
<code>triggerDispute</code>	Step 6	Leaves the optimistic path and initializes the dispute game. In the measured contracts, this may include dispute deployment.
<code>respondChallenge</code>	Step 7b	Buyer response during the binary search over the computation trace.
<code>giveOpinion</code>	Step 7c	Vendor opinion on the buyer response; updates the search interval.
<code>submitCommitment</code> <code>submitCommitmentLeft</code> <code>submitCommitmentRight</code>	Step 8	Local verification of a circuit gate, boundary value, or final comparison using authenticated data.
<code>finalize</code>	Step 9	Ends the current dispute execution or triggers the next protocol consequence according to the sponsor configuration.

Table 2.2: Mapping between implementation terminology and protocol steps.

This mapping is used later in the experimental chapter. Without it, gas tables are easy to misread. For example, a line labelled “dispute cost” may include only Step 8, the whole post-trigger dispute, or also the deployment cost of the dispute contract. This project therefore reports these scopes separately.

2.6 Circuit commitments: h_{ct} , $h_{circuit}$, and h_{pre}

The dispute mechanism relies on commitments that allow the contract to verify small pieces of a large off-chain computation. The implementation uses Merkle-style commitments over fixed-size objects. The precise encoding is discussed later in the report because it also raises security and ambiguity

questions, but the conceptual role of each commitment is as follows.

Commitment	Role in the protocol
h_{ct}	Commitment to the ciphertext blocks. During Step 8, a party can reveal only the block needed for a local gate evaluation, together with a membership proof.
$h_{circuit}$	Commitment to the circuit description. In the generic mode, the contract does not know the whole circuit, so the vendor must authenticate the gate being evaluated.
h_{pre}	Commitment or authenticated value associated with intermediate computation values used during the dispute search. It lets the parties argue about a particular prefix or local position without publishing the full trace on-chain.

Table 2.3: Commitments used to keep the dispute verification local.

The circuit used by the inherited implementation is encoded as a sequence of fixed-size 64-byte gates. A gate may represent, for example, an AES-CTR decryption block, a SHA-256 compression step, a constant, an XOR operation, or a comparison. The contract can evaluate a single gate if it receives the gate encoding, the necessary input values, and proofs that those inputs are consistent with the committed ciphertext or intermediate trace.

The role of $h_{circuit}$ is especially important for understanding the hardcoded SHA256 optimization. In the generic mode, the contract must be convinced that the gate submitted in Step 8 is really the gate committed to by the precontract. This is why the generic Step 8 includes a proof for the circuit commitment, denoted π_1 in the project discussions. In the hardcoded SHA256 mode, the contract can reconstruct the expected gate from public parameters such as the file length, the description hash, and the gate index. In that specialized case, the proof π_1 is no longer needed for the same purpose because the circuit structure is no longer supplied by the vendor.

This optimization does not remove all Step 8 costs. The contract still has to verify the relevant ciphertext or intermediate-value proofs and still has to evaluate the local gate. It only removes the generic circuit-authentication part of Step 8. This distinction explains why the local hardcoded gain can be visible on the Step 8 subcomponent while being diluted in the total cost of a complete dispute.

2.7 ERC-4337 and transaction sponsorship

The implementation also contains a transaction-sponsorship layer inspired by ERC-4337 account abstraction. Instead of requiring each user to send every

transaction directly from a funded externally owned account, the application can package calls as user operations and send them through a bundler and an EntryPoint contract.

This layer is important for usability because it makes it possible to demonstrate sponsored execution: a participant can trigger protocol actions without manually managing gas in the same way as a regular Ethereum transaction. It is also important experimentally because it introduces infrastructure that is not part of the mathematical core of the protocol. A bundler, an EntryPoint, and local deployment scripts can affect latency, failure modes, and the way gas is paid.

For this reason, the report distinguishes two notions of sponsorship:

- *protocol sponsorship*, which is the role of S , S_B , and S_V in the SOX protocol and determines deposits, reimbursements, and dispute incentives;
- *transaction sponsorship*, which is the implementation mechanism that submits calls through an account-abstraction stack.

These notions are related but not identical. The no S -deposit mode simplifies protocol sponsorship in the optimistic phase. It does not automatically remove every piece of transaction infrastructure from the application. Conversely, an ERC-4337 user operation can be used to submit a call whose economic meaning is still governed by the protocol roles and deposits. Keeping this distinction explicit was necessary to avoid confusing gas paid to execute Ethereum transactions with the protocol deposits used to compensate sponsors.

2.8 Security assumptions and fairness goals

The security goal of SOX is best understood operationally. If the vendor behaves honestly, publishes a valid encrypted item, and reveals the correct key, then the vendor should receive the payment unless the buyer can provide valid evidence of a problem. If the buyer behaves honestly and the decrypted item is invalid, unavailable, or inconsistent with the precontract, then the buyer should be able to prevent the vendor from keeping the payment by following the dispute procedure. Sponsors should be reimbursed or penalized according to the rules encoded in the contract.

The implementation relies on the following assumptions.

- The blockchain executes the deployed smart contracts deterministically and makes their state publicly available.
- Deadlines are long enough for honest parties or their sponsors to submit the required transactions.

- The hash functions used for commitments and Merkle proofs are collision resistant for the encoded objects being committed.
- The encryption scheme keeps x confidential before the key is revealed.
- The off-chain computation used to produce ciphertexts, circuits, intermediate values, and proofs is consistent with the on-chain verifier.
- The data needed for a dispute, especially ciphertext blocks and proof artifacts, remains available to the party that must use it.
- The transaction-sponsorship infrastructure, when used, is live enough not to censor honest protocol actions indefinitely.

These assumptions also show the limits of the current prototype. The smart contract can check authenticated local claims, but it cannot force an unavailable backend to serve data, cannot make a bundler include a transaction, and cannot compensate for ambiguous low-level encodings if such encodings allow two interpretations of the same byte string. The latter point became important during the project when reviewing the inherited gate encoding and Merkle construction. The specialized gas optimizations developed in this project do not depend on creating new encodings, but a production-grade version should make those encodings canonical and domain-separated.

The rest of the report builds on these foundations. Chapter 3 describes the inherited implementation and its execution path. Later chapters then explain how the protocol variants were isolated, how the final specialized contracts were implemented, and how the gas and off-chain timings were measured.

Chapter 3

Initial Implementation Analysis

This chapter analyzes the implementation that was inherited at the beginning of the project. The objective is not yet to discuss the optimized variants introduced later, but to explain what the starting system already did, how its components interacted, and which practical limitations motivated the rest of the work. This distinction matters because the current repository now contains both inherited components and new specialized contracts. Unless explicitly stated otherwise, this chapter refers to the initial execution path centered on the original optimistic account, the original dispute account, the Rust/WebAssembly precontract pipeline, the Next.js backend, the local SQLite database, and the Alto/EntryPoint transaction path.

3.1 Repository structure and technology stack

The initial codebase was not a single smart contract repository. It was a full proof of concept combining off-chain computation, on-chain adjudication, a local web application, and transaction-sponsorship infrastructure. The main components are summarized in Table 3.1.

The local development stack is based on a Hardhat chain, Solidity contracts compiled through Hardhat, a Next.js server running on `localhost:3000`, a bundler RPC endpoint running on `localhost:4337/rpc`, and a SQLite database stored under `src/app/db`. This setup is appropriate for a research prototype because it makes the complete execution path visible and reproducible. It is not, however, equivalent to a production deployment: identities are local test accounts, the backend is centralized, and the bundler is a local development service.

The contract compiler configuration also reflects this research-prototype status. The Hardhat configuration enables optimization and compilation through the Solidity intermediate representation, and local networks are

Component	Role in the initial implementation
<code>src/wasm</code>	Rust crate used for cryptographic preprocessing, circuit construction, ciphertext generation, commitments, and native command line tools. It also produces the WebAssembly bundle used by the application.
<code>src/hardhat</code>	Hardhat workspace containing Solidity contracts, deployment scripts, generated artifacts, and tests. The initial path uses an optimistic account and a dispute account, plus libraries for accumulators, commitments, AES, SHA-256, and circuit evaluation.
<code>src/app</code>	Next.js application containing the frontend, API routes, SQLite database access, blockchain helper code, and local file storage endpoints.
<code>bundler-alto</code>	Local Alto bundler used to submit UserOperations to the local chain through an EntryPoint-compatible account-abstraction path.
<code>desktop</code>	Optional Electron shell around the Next.js application. It is useful for local demonstrations and for executing native preprocessing outside the browser.
Root scripts	Shell scripts such as <code>install.sh</code> , <code>deploy-all.sh</code> , and <code>run-alto.sh</code> orchestrate installation, contract deployment, and bundler startup.

Table 3.1: Main components of the inherited implementation.

configured with unlimited contract size. This was necessary because the original dispute contracts and deployer libraries were large. It also means that local deployment success alone cannot be interpreted as mainnet deployability.

3.2 Rust and WebAssembly off-chain pipeline

The most important off-chain component is the Rust crate in `src/wasm`. It implements the preprocessing required before an exchange can be proposed. The native binary `precontract_cli` takes an input file and an optional 16-byte AES key. If no key is given, it samples a random one. It then computes the precontract material used by the rest of the system.

The output of this command line tool contains, in particular:

- the description output encoded as hexadecimal data;
- the ciphertext commitment h_{ct} ;
- the circuit commitment $h_{circuit}$;
- the commitment value c and opening value o ;
- the number of ciphertext blocks and the number of circuit gates;
- paths to the generated ciphertext and circuit files;
- the AES key, returned so that the vendor can later reveal it on-chain.

The same crate contains the circuit construction and evaluation code. The implementation uses a compact V2 circuit representation based on fixed 64-byte gates. The relevant operations include AES-CTR decryption gates, SHA-256 compression gates, constants, XOR operations, and a final comparison gate. This representation is central to the dispute mechanism because it lets the smart contract evaluate a single local gate once the corresponding gate bytes and input values are authenticated.

The Rust code is exposed in two ways. The WebAssembly bundle is used by the web application for interactive use, while native command line tools are used by the backend and by performance experiments. This dual execution path was already important in the inherited system. The browser/WebAssembly path is convenient for demonstrations and moderate files, but native execution is more appropriate for large files because it avoids browser memory and UI constraints.

The inherited proof-generation API was only partial. The route `src/app/api/proofs/compute/route.ts` calls the native `compute_proofs_cli` binary, but the exposed path supports only one dispute state in the current application interface: the state corresponding to `WaitVendorDataRight`. This was sufficient for some local tests and demonstrations, but it did not

provide a complete API for all possible Step 8 cases. This limitation is one reason why later measurements use dedicated test scripts and native benchmark binaries rather than relying exclusively on the web API.

3.3 Frontend, backend, database, and file storage

The user-facing application is implemented with Next.js and a small set of role-oriented components. The user side contains views for creating contracts, accepting precontracts, and following ongoing contracts. The sponsor side contains views for contracts waiting for sponsorship and for dispute sponsorship. The application also has blockchain helper modules for optimistic contracts, dispute contracts, UserOperations, deployment configuration, and contract artifacts.

The backend stores protocol metadata in SQLite. The main table, **contracts**, stores the precontract data needed by the application: item description, commitment opening, buyer and vendor addresses, price, number of blocks, number of gates, commitment, completion and dispute tips, protocol version, timeout delay, algorithm suite, acceptance status, sponsor address, deployed optimistic contract address, and session-key information. A second table, **disputes**, links a contract identifier to buyer-side and vendor-side dispute sponsors and to the deployed dispute contract address.

The encrypted file itself is stored off-chain under the application upload directory. During precontract creation, the backend can receive a multipart upload, call the native `precontract_cli`, copy the generated ciphertext to `src/app/uploads`, and insert the corresponding metadata into SQLite. Alternatively, in Electron mode, the frontend can send precomputed values produced locally.

This design is practical for a proof of concept because it makes the whole system easy to run on one machine. Cryptographically, the backend is not trusted to decide the fairness of the exchange: the smart contract only relies on commitments and proofs. Practically, however, the backend remains an availability and orchestration dependency. If the ciphertext or proof artifacts are not available when needed, the on-chain protocol cannot reconstruct them by itself. This is not a contradiction of the protocol, but it is an important implementation assumption.

The initial application also uses predefined local accounts. Public/private key pairs are hard-coded for the local Hardhat chain, and the UI behaves like a laboratory multi-user interface rather than a wallet-authenticated application. This makes testing much faster, but it must not be confused with a real wallet integration.

3.4 Optimistic smart contract architecture

The initial optimistic phase is implemented by an account-style contract. The central contract in the inherited execution path is `OptimisticSOXAccount.sol`. It combines three responsibilities that are conceptually distinct:

- it stores the exchange parameters and optimistic state;
- it escrows payments, tips, and sponsor funds;
- it also behaves as an account-abstraction compatible account for sponsored calls.

The optimistic state machine contains the states

`WaitPayment`, `WaitKey`, `WaitSB`, `WaitSV`, `InDispute`, `End`.

The normal execution begins in `WaitPayment`. The buyer calls `sendPayment`, after which the contract waits for the vendor to reveal the AES key with `sendKey`. Once the key is available, the buyer can either complete the exchange with `completeTransaction` or move toward a dispute by registering the buyer-side dispute sponsorship. The vendor-side dispute sponsorship then creates or activates the dispute contract and moves the optimistic contract into `InDispute`.

The constructor of the optimistic account receives the `EntryPoint` address, vendor and buyer addresses, price, tips, timeout increment, commitment, number of ciphertext blocks, number of gates, and vendor signer. The inherited design deploys a new optimistic account for each exchange. This is simple and modular: each exchange has its own state, its own balance, and its own account-abstraction context. The cost is that deployment becomes part of the marginal cost of every exchange.

The contract also implements timeout logic. If the expected party does not act before the deadline, `cancelTransaction` redistributes funds according to the current state. This is important for liveness: an honest party should not be blocked forever by a counterparty who stops responding.

3.5 Dispute smart contract architecture

The dispute phase is implemented by a separate dispute account. In the inherited path, the dispute account is deployed when the optimistic contract reaches the dispute stage. The main contract is `DisputeSOXAccount.sol`, supported by several verifier and evaluator libraries.

The dispute contract keeps the state of the binary search. Its states cover buyer challenges, vendor opinions, vendor data submissions, completion, cancellation, and final termination. In the Solidity implementation these

states are named `ChallengeBuyer`, `WaitVendorOpinion`, `WaitVendorData`, `WaitVendorDataLeft`, `WaitVendorDataRight`, `Complete`, `Cancel`, and `End`. The buyer submits a response with `respondChallenge`. The vendor answers with `giveOpinion`, which updates the search interval. When the search reaches a local position or a boundary case, the vendor submits authenticated local data through one of the `submitCommitment` calls.

The generic Step 8 verification is expensive because it does several things at once. It opens the commitment to recover $h_{circuit}$ and h_{ct} , checks Merkle-style proofs for the relevant circuit gate and values, evaluates the local V2 gate, and checks the consistency of the resulting accumulator value. The contract therefore depends on helper contracts and libraries for accumulator verification, commitment opening, V2 circuit evaluation, AES-CTR evaluation, SHA-256 evaluation, and simple operations.

This architecture follows the intended optimistic principle: the whole file and the whole circuit are never evaluated on-chain in the honest case. Nevertheless, once a dispute starts, the deployment and local verification costs are substantial. The initial implementation also treats the circuit generically, so the contract must be given the 64-byte gate encoding and the proof that this gate belongs to $h_{circuit}$. This generic design is flexible, but it is not optimal when the description is known to be `SHA256` and the circuit structure is determined by the input length.

3.6 ERC-4337, Alto bundler, and EntryPoint integration

The inherited implementation does not only implement protocol sponsorship at the economic level. It also integrates a transaction-sponsorship path inspired by ERC-4337. The local stack deploys or configures an `EntryPoint`-compatible contract and starts an Alto bundler. The frontend builds `UserOperations`, sends them to the bundler RPC endpoint, and waits for the corresponding `UserOperation` receipt.

Both the optimistic account and the dispute account expose account-style functionality: signature validation, nonces, execution helpers, and `EntryPoint` deposit management. This allows actions such as payment, key release, challenge response, and dispute evidence submission to be executed through the account-abstraction path. In the local prototype, this is useful because it demonstrates how a participant can interact with the protocol without manually sending every transaction as a regular externally owned account transaction.

This integration also introduces additional assumptions. The bundler must be running, the `EntryPoint` address must match the deployed configuration, and the `UserOperation` must be accepted and included. From the point of view of protocol analysis, the bundler is not part of the original fairness

proof. It is an implementation layer that affects how gas is paid and how liveness is achieved in practice. This is why later chapters separate protocol variants from transaction-sponsorship infrastructure.

3.7 Initial end-to-end execution flow

The initial end-to-end flow validated during the first phase of the project is shown in Table 3.2. It starts before any smart contract call, with the vendor creating the precontract material, and ends either in optimistic completion or in a dispute contract.

Order	Location	Action
1	Rust / backend	The vendor uploads or selects a file. The preprocessing pipeline encrypts it, builds the circuit, computes h_{ct} , $h_{circuit}$, and the commitment, and stores the generated artifacts.
2	SQLite / uploads	The backend records the precontract metadata and stores the encrypted file off-chain. The buyer can see pending precontracts addressed to its public key.
3	Frontend	The buyer accepts the precontract. In the initial application this is a database-level acceptance action before the on-chain optimistic account is deployed or sponsored.
4	Solidity / Hardhat	The optimistic account is deployed with the precontract parameters and sponsor-related funds.
5	Optimistic account	The buyer pays into the contract, then the vendor sends the AES key. If the buyer accepts the decrypted item, the exchange completes.
6	Dispute path	If the buyer complains, buyer-side and vendor-side dispute sponsorship are registered and a dispute account is deployed. The dispute then proceeds through challenge rounds and Step 8 verification.

Table 3.2: Initial end-to-end execution flow observed in the inherited implementation.

This flow was essential to reconstruct before changing the protocol vari-

ants. Several gas costs depend on decisions made outside the Solidity functions themselves. For example, a measurement can include contract deployment, EntryPoint deposit funding, database-level precontract acceptance, dispute deployment, or only a single function call. The initial analysis therefore established the discipline used later in the report: every measurement must say which part of this flow it covers.

3.8 Limitations of the initial implementation

The inherited implementation was already a complete and useful proof of concept, but it had several limitations for the goals of this project.

Deployment cost was paid per exchange. The initial architecture deploys an optimistic account for each exchange and a dispute account when a dispute starts. This makes the state isolation simple, but it makes deployment cost part of the marginal cost of the protocol. For gas optimization, this is a major issue: a large fixed deployment cost can dominate the cost of an honest exchange.

The optimistic and dispute accounts were too general. The initial contracts were designed to support the full sponsored protocol. This is useful for correctness and demonstration, but it means that exchanges pay for code paths that are not needed in simpler variants. For example, the normal sponsored case, no S -deposit, $S = B$, $S = V$, self-sponsored disputes, and hardcoded SHA256 do not all need the same storage fields and runtime branches. This observation motivated the later move from monolithic variant support to specialized contracts.

The generic dispute verifier pays for circuit flexibility. In the generic dispute path, Step 8 authenticates a supplied 64-byte gate against $h_{circuit}$. This is necessary for arbitrary circuits, but it is redundant when the circuit is fully determined by public parameters. The `desc = SHA256` case is the main example: once the input length is fixed, the sequence of SHA-256 circuit gates is determined. This created the opportunity for the hardcoded SHA-256 dispute mode implemented later.

The backend remains an orchestration and availability dependency. The encrypted file and auxiliary artifacts are stored off-chain. The commitments prevent the backend from silently changing the data without detection, but they do not make unavailable data appear. A robust deployment would need a clearer availability layer, for example replicated storage or explicit user-side artifact management.

The web proof-generation path was incomplete. The backend API for proof computation exposed only a limited part of the dispute proof generation flow. Complete experimental coverage therefore required Hardhat tests and native Rust benchmark tools rather than relying only on the interactive web interface.

The identity model was a laboratory model. The application uses predefined Hardhat accounts and a local mapping from public addresses to private keys. This is appropriate for reproducibility, but it does not provide real wallet authentication or production-grade key management.

Large files required the native path. The Rust/WebAssembly architecture made preprocessing portable, but the browser path was not the right environment for very large files. For large-file measurements, especially the 1 GiB experiments discussed later, the native Rust command line tools were the reliable execution path.

These limitations do not invalidate the initial implementation. On the contrary, they define the engineering problem addressed by this project. The inherited system already made the protocol executable end-to-end; the remaining question was how to restructure it so that the important protocol variants could be measured at the correct scope and with lower gas cost.

Chapter 4

Methodology and Experimental Discipline

4.1 Project methodology

The project methodology was deliberately incremental. The implementation was already large when the project started, and changing protocol logic before understanding the real execution path would have made the gas measurements unreliable. The work was therefore organized around four principles:

1. first reproduce the inherited implementation end-to-end;
2. then map implementation actions to protocol steps;
3. only then implement variants and optimizations;
4. finally measure each variant with explicit accounting scopes.

The first phase was diagnostic. The goal was to make the full local stack run: Hardhat node, contract deployment scripts, Rust and WebAssembly artifacts, SQLite database, Next.js application, Alto bundler, and optional Electron shell. This phase was not only installation work. It established which code paths were actually used by the application and which contracts were only present as historical or testing artifacts.

The second phase was protocol mapping. The paper describes the protocol as a sequence of steps, while the implementation exposes functions, API routes, UI actions, and benchmark helpers. A gas number is meaningful only if the measured code can be mapped back to a protocol action. This mapping was especially important for the dispute phase, where a single informal label such as “dispute cost” can refer to deployment, trigger, challenge rounds, Step 8 verification, finalization, or all of them together.

The third phase was implementation. The first implementation strategy was conservative: support variants by extending the existing contracts. This was useful for functional validation, but it produced monolithic contracts with many branches. After the first gas measurements and feedback from the supervisor, the implementation strategy was revised: variant selection should happen off-chain or at deployment time, and the smart contract used for one exchange should not carry unrelated modes. This led to the specialized final implementation described in later chapters.

The fourth phase was measurement and consistency checking. Several early measurements were individually correct but not directly comparable because they mixed scopes. The final methodology therefore treats measurement scope as part of the experimental result. Every reported number must specify which implementation was measured, which protocol steps were included, and whether the cost is one-time infrastructure cost or marginal per-exchange cost.

4.2 Reproducibility strategy

The reproducibility strategy has two layers. The first layer is executable reproducibility: the repository contains scripts and tests that can recreate the local stack and rerun the measurements. The second layer is accounting reproducibility: the report explains exactly how to recombine the measured components.

The local stack is started with the root scripts documented in the repository. In the normal development workflow, a Hardhat node is started first, `deploy-all.sh` deploys the contract stack and writes deployment information, `run-alto.sh` starts the Alto bundler, and `npm run dev` starts the Next.js application. This is the same environment used for interactive demonstrations.

For gas measurements, the main source of reproducibility is the Hardhat test suite under `src/hardhat/test/performance`. These tests deploy their own local contracts, execute transactions, and read gas from actual transaction receipts. The tests avoid relying on manual UI actions for the final numbers. The UI is useful to demonstrate the proof of concept, but scripted tests are more appropriate for repeatable accounting.

The benchmarks also control cryptographic inputs whenever possible. Small and medium files are generated deterministically inside the test code. The AES key is fixed in gas tests so that the same circuit and commitments can be reproduced. Large-file tests create temporary input files and validate the resulting number of ciphertext blocks and gates before running the dispute scenario. Temporary artifacts are removed after the test when the benchmark finishes.

This strategy does not mean that all timings are machine-independent.

Gas measurements are deterministic for a fixed compiler, contract version, and local EVM configuration. Off-chain timings depend on the machine, file system, and runtime environment. The report therefore separates gas results from wall-clock timings and never uses local CPU time as a substitute for on-chain gas.

4.3 Mapping code functions to protocol steps

The implementation uses names that are convenient for Solidity and frontend code, but they do not always match the step names used in the paper. The mapping in Table 4.1 is the one used throughout the measurements.

Implementation action	Protocol step	Measurement consequence
<code>triggerDispute</code>	Step 6	Starts the dispute path. In the measured contracts this action may also deploy or create the dispute account, so it must not be interpreted as a small state transition only.
<code>respondChallenge</code>	Step 7b	Buyer-side answer to the current challenge. Together with <code>giveOpinion</code> , it forms one challenge round.
<code>giveOpinion</code>	Step 7c	Vendor-side answer that updates the binary-search interval.
<code>submitCommitment</code> variants	Step 8	Local verification step. Depending on the state, it verifies a generic gate, a left boundary condition, or a right/final condition.
<code>finalize</code> or final state transition	Step 9	Ends the current dispute execution or applies the protocol consequence determined by the sponsor configuration.

Table 4.1: Mapping used to interpret implementation-level gas measurements.

This mapping also explains why the number of Step 7–8 iterations matters. In normal sponsored disputes, the protocol may require more than one execution of the challenge and verification loop because of sponsor rotation. In self-sponsored cases such as $S_B = B$ and $S_V = V$, the same party also plays the relevant sponsor role, so unnecessary repetitions can be removed.

The methodology therefore reports both per-call costs and complete dispute costs when needed.

4.4 Gas accounting scopes

The most important methodological decision was to define gas scopes before comparing implementations. The project uses the scopes in Table 4.2.

Scope	Meaning
One-time infrastructure	Gas paid once to deploy reusable components such as verifier libraries, dispute deployers, implementation contracts, or the factory. These costs are not paid again for every exchange if the infrastructure is reused.
Marginal optimistic exchange	Gas paid for one honest exchange after reusable infrastructure already exists. Depending on the variant, this includes clone creation or direct deployment and calls such as payment, key release, and completion.
Optimistic path before dispute	Gas paid to reach the point where a dispute can be triggered. This includes optimistic deployment or creation and the payments or deposits needed before Step 6.
Dispute trigger / initialization	Gas paid by <code>triggerDispute</code> . In the measured architecture this may include deployment or creation of the dispute contract.
Dispute after trigger	Gas paid after the dispute contract exists: challenge rounds, Step 8 verification, and finalization.
Single Step 8 call	Gas paid by one <code>submitCommitment</code> -type call only. This scope isolates local verifier changes, for example the removal of π_1 in the hardcoded SHA-256 mode.
End-to-end dispute path	Combination of optimistic setup, dispute trigger, post-trigger dispute execution, and finalization for a full disputed exchange.

Table 4.2: Gas accounting scopes used in the report.

This separation avoids two common mistakes. The first mistake is comparing a one-time infrastructure deployment to a per-exchange deployment. This would make clone-based architectures look artificially expensive at setup time or artificially cheap at exchange time. The second mistake is comparing a single Step 8 optimization with a complete dispute. For example, removing

the generic circuit proof π_1 can save gas locally in Step 8, but the total dispute also includes trigger, challenge rounds, proof verification for other commitments, gate evaluation, and finalization.

Gas is always taken from transaction receipts, not from informal estimates. In the tests this is done by waiting for the transaction receipt and reading the `gasUsed` field. This matters because storage writes, warm/cold accesses, deployment bytecode, and repeated calls can make the real gas different from a rough estimate.

4.5 Off-chain timing methodology

Off-chain timings are measured separately from gas. They are used to evaluate the practical cost of preparing precontracts, computing dispute witnesses, and generating the data needed for challenge rounds. They do not represent blockchain confirmation time and they are not converted into gas.

Two execution paths are used:

- the WebAssembly path, used by the application and by several tests for small or medium inputs;
- the native Rust command line path, used for large-file experiments and for the 1 GiB hardcoded SHA-256 dispute benchmark.

The native path is essential for large files. The browser path is useful for a proof-of- concept demonstration, but it is not the right benchmark environment for 1 GiB inputs. The native `precontract_cli` measures the cost of reading the input file, encrypting it, building the circuit, computing commitments, and writing the ciphertext and circuit artifacts. The native `hardcoded_sha256_dispute_cli` measures the off-chain generation of challenge-round values and the final Step 8 witness data for the hardcoded SHA-256 scenario.

The timing methodology is conservative in interpretation. When a benchmark reports the time needed to prepare a precontract, this is primarily the vendor-side preparation time in the current implementation. A fully protocol-level performance table may also account for the buyer-side verification work, even if the current application does not perform an identical recomputation before acceptance. Similarly, challenge-response generation is measured for the party producing the response, while a complete protocol accounting may also include the counterparty's verification work. The report therefore distinguishes measured implementation time from conservative protocol-level estimates.

4.6 Benchmarking environment

The on-chain benchmarks are run on a local Hardhat network. The Solidity compiler version is 0.8.28, with optimization enabled, compilation through the intermediate representation, and optimizer runs set to one. The local network is configured to allow large contracts, which is useful for testing and for comparing architectures, but it is not a mainnet guarantee. When mainnet deployability matters, contract bytecode size must be checked separately.

Most gas tests deploy a mock EntryPoint or local EntryPoint-compatible contract rather than depending on a live external bundler. This makes the measurements deterministic and keeps the focus on contract gas rather than on bundler behavior. The interactive application, by contrast, uses the local Alto bundler and the configured EntryPoint address. Both paths are useful, but they answer different questions:

- Hardhat tests answer: how much gas does this contract path consume?
- The full application answers: can the proof of concept be executed through the UI and account-abstraction infrastructure?

The benchmarks report gas units only. They do not multiply by an ETH price or by a gas price, because such conversions are time-dependent and would not be stable. They also do not claim to model L1 congestion, bundler policies, or wallet UX. Those topics are important for deployment, but the experimental goal of this project is to compare protocol variants under a controlled local EVM configuration.

4.7 Validation strategy

Validation was performed at several levels.

Functional validation. The application was run end-to-end on the local stack. This validated the practical path from precontract creation to buyer acceptance, contract sponsorship or deployment, buyer payment, vendor key release, and optimistic completion. This is the path used for demonstrations.

Contract-level validation. Hardhat tests exercise the optimistic and dispute contracts directly. They check state transitions, role restrictions, deployment paths, challenge rounds, and Step 8 submissions. The performance tests also include assertions on expected values such as the number of blocks, number of gates, and number of challenge rounds for the generated circuit.

Variant validation. Separate tests cover the final specialized architecture, including the normal case, no S -deposit, $S = B$, $S = V$, self-sponsored dispute execution, and hardcoded SHA-256. The purpose is not only to obtain gas numbers but also to ensure that the variants still execute the intended protocol path.

Same-scope comparison. Several tests were written specifically to avoid comparing incompatible measurements. For example, one set of benchmarks measures the current implementation with the same high-level scope as the previous implementation report, while another set compares monolithic and specialized variants under the same file size and dispute path. This same-scope discipline was added because early tables were difficult to interpret when they mixed architectural versions.

Large-file validation. The 1 GiB hardcoded SHA-256 benchmark validates the large-file path with native CLIs. The test checks that the generated number of blocks and gates matches the expected values, executes all challenge rounds for the chosen path, submits the final Step 8 witness, and finalizes the dispute. This validates the feasibility of the large-file pipeline in the measured scenario.

This validation strategy is practical rather than formal. It increases confidence that the implementation and measurements correspond to the intended protocol flows, but it does not replace a full proof of security. In particular, the inherited gate encoding and Merkle domain-separation questions identified later remain protocol-engineering issues for a future clean version.

Chapter 5

Protocol Variants and Design Choices

5.1 Motivation for protocol variants

The inherited implementation supported the main optimistic and dispute flows, but it did so through contracts that were close to a “one-size-fits-all” design. This was convenient for experimentation, because a single contract family could execute many scenarios. It was less suitable for gas optimization. A contract that supports every role configuration must keep state variables, checks, branches, and deployment bytecode that are irrelevant for many exchanges. The central design question of the project therefore became: which variants are protocolically meaningful, and which of them justify a separate implementation path?

The answer is not simply the Cartesian product of all possible role equalities. Some choices are known at precontract time, before the optimistic contract is created. Other choices are only known if the optimistic path fails and the parties enter the dispute path. The final implementation separates these two moments. Precontract-time variants select the optimistic contract to deploy or clone. Dispute-time variants select the appropriate dispute contract when the dispute sponsors actually appear.

Table 5.1 summarizes the classification used in the implementation. The table is also the convention used in the rest of the report.

The guiding principle is that a variant is worth implementing separately only if it changes the cost, risk, or latency of a protocol path. Exotic role equalities that do not change the execution path are still valid theoretical cases, but they do not necessarily deserve their own contract. This choice keeps the implementation close to the supervisor feedback: optimize the cases that matter for measurement, and avoid paying gas for branches that are not used in the selected mode.

Variant	Decision time	Main effect
Normal sponsored SOX	Precontract / deployment	Baseline model with an optimistic sponsor S and potentially distinct dispute sponsors S_B and S_V .
<code>no_S_deposit</code> / <code>simSOX</code>	Precontract / deployment	Removes the optimistic sponsor deposit and the corresponding reimbursement logic from the optimistic phase.
$S = B$	Precontract / deployment	Makes the buyer the optimistic sponsor and allows the implementation to fuse the buyer payment with the initial creation step.
$S = V$	Precontract / deployment	Makes the vendor the optimistic sponsor. The buyer payment remains a later optimistic action in the current implementation.
$S_B = B$	After buyer dissatisfaction	Makes the buyer its own dispute sponsor and makes the Step 4–5 fusion trivial.
$S_V = V$	After buyer dissatisfaction	Makes the vendor its own dispute sponsor. Together with $S_B = B$, it removes the need for a sponsor-rotation repetition in Step 9.
Hardcoded SHA256	Precontract / deployment	Replaces the generic circuit-opening check for $h_{circuit}$ in Step 8 by an on-chain reconstruction of the expected SHA256 circuit gate.

Table 5.1: Protocol variants retained for implementation and measurement.

5.2 Normal sponsored SOX

The normal sponsored variant is the reference point. It keeps the full role structure of the SOX protocol: vendor V , buyer B , optimistic sponsor S , buyer-side dispute sponsor S_B , and vendor-side dispute sponsor S_V . In this mode, the optimistic sponsor creates the exchange contract and deposits the funds needed by the optimistic mechanism. The buyer then pays, the vendor releases the key, and the exchange either completes or moves toward a dispute.

In implementation terms, the final codebase keeps a normal clone implementation for factory-based measurements, while the interactive application keeps the direct normal account path for compatibility with the existing ERC-4337 flow. In both cases, the normal path starts in state `WaitPayment`. This means that Step 1 and Step 2 remain distinct in the normal sponsored path: the sponsor creates and funds the exchange first, and the buyer later sends the payment.

Keeping this variant is important for two reasons. First, it is the closest implementation of the sponsored protocol as inherited from the original codebase. Second, it provides a fair baseline against which all simplified modes can be compared. A simplified mode is only useful if it is clear which part of the normal sponsored logic it removes.

The normal path also remains the fallback for disputes with external sponsors. When the buyer-side and vendor-side dispute sponsors are not both self-sponsored, the optimistic clone deploys the normal dispute contract. This preserves the generic Step 9 behavior, including the possibility of repeated Step 7–8 executions when the protocol logic requires a new sponsor configuration.

5.3 No S -deposit and simSOX

The `no_S_deposit` mode is the simplified optimistic mode used in the report under the name `simSOX`. Its goal is precise: remove the initial sponsor deposit from the optimistic phase. It should not be confused with a complete redesign of transaction delivery, and it should not be described as merely “funding the `EntryPoint` differently”. At the protocol level, the important point is that there is no optimistic S -deposit to reimburse.

This distinction matters because the transaction layer and the protocol economics are separate. ERC-4337 or a bundler can be used to deliver a transaction, but the `no_S_deposit` variant is about the economic state of the SOX contract itself. In the specialized clone implementation, `OptimisticSOXCloneNoSDeposit` enforces this by requiring zero value at initialization:

- the factory function `createNoSDeposit` rejects any initial value;

- the clone starts in `WaitPayment`, like the normal path;
- the buyer payment required by `sendPayment` is only the agreed price;
- the sponsor tip recorded after payment is set to zero.

Thus, `simSOX` is not a claim that no one pays gas. It is the case where the optimistic part of the protocol no longer contains an S -deposit whose purpose is to reimburse external sponsoring. This makes it a useful comparison point for honest exchanges, because it removes a piece of accounting that is unnecessary when participants pay their own transaction costs in the optimistic phase.

The mode is selected before deployment. The frontend and backend expose it as a precontract variant, and the factory chooses the corresponding implementation. This is important: the contract does not carry both the normal sponsor-deposit logic and the no-deposit logic at runtime. The selected clone contains the behavior needed for that mode.

5.4 $S = B$: sponsor is buyer

The $S = B$ variant is the cleanest precontract-time self-sponsoring case. Since the buyer is also the optimistic sponsor, the two first steps of the optimistic protocol can be fused in the implementation. Conceptually, Step 1 is the creation and sponsor deposit by S , while Step 2 is the buyer payment. If $S = B$, the same actor is responsible for both actions, so there is no need to expose a state where a third-party sponsor has created an exchange but the buyer has not yet committed funds.

The specialized contract `OptimisticSOXCloneSponsorIsBuyer` implements this fusion. During initialization, it requires the sponsor to be the buyer and requires enough value to cover the buyer payment, the completion tip, and the sponsor fees. It then records the buyer deposit immediately and starts the exchange in state `WaitKey`. Consequently, the buyer does not need to call `sendPayment` as a separate transaction.

This variant addresses the risk identified during the project discussion: if Step 1 and Step 2 are separated, a sponsor may deploy and fund a contract that is not used because the buyer never pays. In the $S = B$ case, the implementation avoids this intermediate state. The buyer's economic commitment and the creation of the exchange happen together.

The implementation also enforces the role equality on-chain. The factory function `createSponsorIsBuyer` requires `msg.sender` to be the configured buyer, and the clone itself requires the sponsor address to match the buyer address. This avoids depending only on frontend selection for a security-relevant role equality.

The $S = B$ mode is distinct from `no_S_deposit`. In `simSOX`, the optimistic sponsor deposit is removed. In $S = B$, the sponsor role still exists,

but it is played by the buyer and its deposit is provided together with the buyer payment. The two modes therefore answer different questions and are measured separately.

5.5 $S = V$: sponsor is vendor

The $S = V$ variant makes the vendor the optimistic sponsor. Like $S = B$, it is determined at precontract time and therefore belongs to the optimistic factory selection. Unlike $S = B$, however, it does not make the Step 1–2 fusion trivial. The buyer is still a different party and still has to send the payment after the optimistic contract has been created.

The dedicated $S = V$ optimistic clone implements this case. Its initialization requires the sponsor to be the vendor, requires the sponsor-fee deposit, and then starts in state `WaitPayment`. The factory function `createSponsorIsVendor` also requires the caller to be the configured vendor. The effect is therefore narrower than in $S = B$: it removes the external optimistic sponsor role, but it does not remove the buyer payment transaction.

This variant was kept because it is a meaningful protocol case and because it was explicitly requested in the revised scope. It is also useful analytically: it separates the question “what happens if the optimistic sponsor is not external?” from the stronger question “can Step 1 and Step 2 be fused?” In the current implementation, the answer is yes for $S = B$ and no for $S = V$.

There is a possible future optimization for $S \neq B$, discussed later in the report: move the loading of the contract or precontract data to Step 2 and require a signature from S . Then B would bring the payment, the precontract data, and S ’s approval in one transaction. That idea is not part of the implemented $S = V$ mode because it changes the deployment architecture more deeply and interacts with ERC-4337 validation.

5.6 $S_B = B$ and $S_V = V$: self-sponsored disputes

The equalities $S_B = B$ and $S_V = V$ belong to a different category. They are not selected when the precontract is created. They become relevant only after the buyer indicates that it is not satisfied and the protocol enters the path that can lead to a dispute.

The first important case is $S_B = B$. In the paper’s step structure, Step 4 is the buyer announcing dissatisfaction, and Step 5 is the buyer-side dispute sponsor registering and depositing. If $S_B = B$, these two actions can be naturally fused: the buyer both expresses dissatisfaction and supplies the buyer-side dispute sponsor deposit. The clone base provides `sendBuyerSelfDisputeSponsorFee` for this self-sponsored case.

For an external S_B , the same fusion is not automatically safe. An external sponsor should not be able to pretend that the buyer is unhappy.

The implementation therefore also contains an authorization-based entry point for external buyer-side dispute sponsors. The authorization hash binds the buyer’s statement to the chain id, the optimistic contract address, the buyer, the chosen dispute sponsor, and the commitment. The external S_B must provide a signature from the buyer before the contract accepts the deposit. This directly addresses the concern that a third party could otherwise trigger the buyer-unhappy path without the buyer’s consent.

The second important case is $S_V = V$. When both $S_B = B$ and $S_V = V$ hold, the dispute does not need the same sponsor-rotation behavior as the fully sponsored case. The losing party is already the relevant economic party. The implementation therefore selects the self-sponsored dispute deployer when the buyer dispute sponsor is the buyer and the vendor dispute sponsor and signer are the vendor. The resulting dispute account uses the self-sponsored Step 9 logic.

The self-sponsored dispute account specializes Step 9. Instead of re-launching or rotating the dispute loop, `handleStep9` moves directly to the appropriate terminal direction: `Cancel` when the vendor loses, and `Complete` otherwise. This is the source of the expected reduction in full-dispute cost: the optimization is not a small local saving in one function, but the removal of unnecessary repeated executions of the Step 7–8 loop.

This design also explains why $S_B = B$ and $S_V = V$ are not implemented as separate precontract factory modes. They are dispute-time facts. The same optimistic exchange may later use either a normal dispute contract or a self-sponsored dispute contract depending on who actually registers as S_B and S_V . Treating them as runtime dispute choices keeps the optimistic contracts smaller and avoids prematurely committing to a dispute path that may never be used.

5.7 Hardcoded SHA256 circuit variant

The hardcoded SHA256 variant targets a different part of the protocol. It is not a sponsoring variant. It is a circuit-verification variant for the case where the description is exactly $desc = \text{SHA256}(x)$. In the generic circuit mode, the dispute contract must verify that the gate opened in Step 8 belongs to the committed circuit. This requires $h_{circuit}$, the gate bytes, and the Merkle proof usually denoted π_1 .

For $desc = \text{SHA256}$, the circuit structure is determined by public metadata: the plaintext length, the number of ciphertext blocks, the description hash, and the AES-CTR IV. The implementation can therefore use a dedicated contract that reconstructs the expected gate on-chain. This removes the need for the vendor to provide the circuit gate bytes as an authoritative description and removes the need to verify π_1 against $h_{circuit}$.

The hardcoded path is fixed at optimistic deployment time through

5.8. VARIANTS KEPT, VARIANTS DISCARDED, AND RATIONALE43

`OptimisticSOXAccountHardcodedSHA256`. The constructor stores the hardcoded metadata: the description hash, the plaintext length, and the ciphertext IV. It also checks that the number of blocks and the number of gates are consistent with the hardcoded SHA-256 layout. The choice is therefore not made by the vendor during Step 8. A generic exchange cannot become hardcoded during a dispute.

The dispute side is handled by `DisputeSOXAccountHardcodedSHA256` and `HardcodedSha256CircuitLib`. In the strict hardcoded tests, the Step 8 call passes `gateBytes = 0x` and an empty `proof1`. The dispute contract ignores these generic circuit-opening inputs and reconstructs the expected gate bytes itself. It then uses the same evaluator and accumulator checks needed to verify the values involved in the gate.

This distinction is important for security. The hardcoded contract determines the gate structure g_i , including the expected opcode and source indices. The vendor may still provide witness values needed to evaluate the gate, because the smart contract does not store the entire ciphertext or all intermediate values. These witness values are not trusted blindly: they are checked against the relevant commitments and Merkle proofs before the gate result is accepted.

The hardcoded optimization is therefore local to the circuit-membership part of Step 8. It does not remove the rest of the dispute work. The contract still has to evaluate the selected gate, verify accumulator proofs for ciphertext or previous values, and execute the normal dispute state machine. This is why later gas tables distinguish between the cost of a single Step 8 call and the cost of a complete dispute.

5.8 Variants kept, variants discarded, and rationale

The final implementation keeps the variants that change a meaningful cost component. It does not implement every theoretically possible role equality as a separate contract. This is a deliberate design decision rather than an omission.

The resulting architecture has three levels of specialization. First, the frontend and backend choose a precontract variant among normal, `simSOX`, $S = B$, and $S = V$. Second, the factory deploys the corresponding minimal optimistic clone, so unused optimistic branches are not carried into every exchange. Third, if a dispute occurs, the optimistic contract selects the normal or self-sponsored dispute deployer based on the actual dispute sponsors. The hardcoded SHA-256 path is kept as a dedicated circuit-specialized path because its optimization concerns Step 8 rather than the optimistic sponsor logic.

This organization is the foundation for the implementation architecture in the next chapter. The important point is that the final design does not

Case	Status	Rationale
Normal sponsored SOX	Implemented	Required baseline for comparison with the inherited sponsored architecture.
<code>no_S_deposit</code> / <code>simSOX</code>	Implemented	Removes the optimistic sponsor deposit and isolates the cost of the simplified honest path.
$S = B$	Implemented	Gives a concrete and safe Step 1–2 fusion because the buyer and optimistic sponsor are the same actor.
$S = V$	Implemented	Removes the external optimistic sponsor role while keeping the buyer payment as a separate action.
$S_B = B$ and $S_V = V$	Implemented as dispute-time specialization	Avoids unnecessary Step 9 repetitions when both dispute sponsors are the corresponding parties.
Hardcoded SHA256	Implemented as a dedicated path	Removes $h_{circuit}$ and π_1 from the dedicated SHA-256 Step 8 verification path.
All 12 role-combination cases as separate optimistic contracts	Not implemented separately	Several combinations differ only after Step 4 or only in accounting labels. Encoding all combinations as optimistic contracts would increase code size without a proportional measurement benefit.
Cases such as $S_B = V$ or $S_V = B$	Documented but not specialized	These edge cases are possible role equalities, but they were not expected to change the main gas evaluation in the retained scenarios.
Arbitrary hardcoded descriptions	Not implemented	The project only implements the dedicated <code>desc = SHA256</code> layout. Other descriptions would need their own circuit-specific verifier.

Table 5.2: Retained and non-retained variant cases.

5.8. *VARIANTS KEPT, VARIANTS DISCARDED, AND RATIONALE*⁴⁵

try to make a single smart contract smarter. It tries to make each deployed contract narrower. In a gas-sensitive protocol, that distinction is essential.

Chapter 6

Implementation Architecture

6.1 From monolithic contracts to specialized contracts

The architectural work of the project followed a useful negative result: adding all variants to one contract was not a good gas optimization strategy. The first monolithic variant implementation extended the existing optimistic and dispute contracts with additional flags for `no_S_deposit`, $S = B$, $S = V$, self-sponsored disputes, and the hardcoded `SHA256` case. This made functional testing possible quickly, but it also produced a monolithic design. The bytecode contained several mutually exclusive branches, so each deployment paid for code that would never be executed by the selected mode.

The final architecture therefore moved from a monolithic contract to a split architecture. The selected mode is decided before deployment whenever the mode is known at precontract time. The contract deployed for one exchange then contains only the logic needed for that exchange. Dispute-specific choices are delayed until the dispute sponsor roles are known, because $S_B = B$ and $S_V = V$ are not precontract-time facts.

The resulting implementation is not a rewrite of the whole application. It preserves the existing Rust precontract logic, the Next.js/Electron user interface, the SQLite storage, the Hardhat test environment, and the ERC-4337-compatible normal path. The main architectural change is the introduction of narrower Solidity components around the existing execution flow.

This split is the key difference between the final implementation and the earlier monolithic version. The protocol did not become less expressive; rather, the implementation stopped carrying unrelated modes inside the same deployed bytecode.

Layer	Responsibility
Frontend and backend	Select and persist the precontract variant, validate role-dependent choices, and pass the selected mode to the deployment layer.
<code>SOXFactory</code>	Deploy minimal optimistic clones for specialized precontract modes when reusable infrastructure is already available.
Optimistic account implementations	Encode the state machine for the honest path and the transition to dispute, with variant-specific deposit logic.
Dispute deployers	Select the normal, self-sponsored, or hardcoded dispute account when a dispute is triggered.
Dispute accounts and verifier libraries	Execute Step 7–9, verify commitments, evaluate gates, and implement hardcoded <code>SHA256</code> reconstruction when selected.
Native Rust CLI tools	Generate precontract artifacts and large-file dispute witnesses outside the browser.

Table 6.1: Main layers of the final implementation architecture.

6.2 Frontend and backend mode selection

The precontract-time variants are exposed to the application through a small TypeScript module, `src/app/lib/protocol-variants.ts`. It defines four user-facing modes: `normal`, `no_S_deposit`, `S_equals_B`, and `S_equals_V`. The labels shown in the interface are deliberately simpler: “normal”, “simSOX (no S-deposit)”, “S = B”, and “S = V”.

The vendor selects the mode when creating a precontract in `src/app/components/user/NewContractModal.tsx`. The selected value is sent to `/api/precontracts` either as multipart form data in the web mode or as JSON in the Electron mode. The backend normalizes the value before inserting it into SQLite. The `contracts` table therefore contains a `precontract_variant` column, with `normal` as the default for backward compatibility.

The same value is later used by the sponsor-facing view. In `src/app/components/sponsor/SponsorContractsListView.tsx`, the interface displays the mode and enforces the obvious role constraints before deployment:

- if the mode is $S = B$, the deployer must be the buyer;
- if the mode is $S = V$, the deployer must be the vendor;
- otherwise, the sponsor can be selected as in the normal flow.

These frontend checks are usability checks, not the security boundary. The Solidity factory and the specialized contracts repeat the relevant role checks on-chain. This duplication is intentional. It gives a clear error to the user early, while still ensuring that a malformed API request cannot deploy a contract under an invalid role equality.

The hardcoded `SHA256` circuit variant is not exposed as a normal drop-down option in the main precontract modal. In the current codebase it is a dedicated deployment and benchmark path, enabled by passing `hardcodedSha256Circuit` metadata to the blockchain deployment helper. This keeps the user-facing mode selection focused on the three sponsoring simplifications that were requested for the interface: `simSOX`, $S = B$, and $S = V$.

6.3 SOXFactory and clone-based optimistic contracts

The contract `src/hardhat/contracts/SOXFactory.sol` is the main on-chain component of the clone architecture. It stores four immutable implementation addresses, one for each optimistic mode: normal, no-deposit, sponsor-is-buyer, and sponsor-is-vendor. For each new exchange, the factory creates an EIP-1167-style minimal proxy and calls `initialize` on the clone.

The factory exposes one creation function per precontract mode:

- `createNormal` for the normal sponsored clone;
- `createNoSDeposit` for `simSOX`;
- `createSponsorIsBuyer` for $S = B$;
- `createSponsorIsVendor` for $S = V$.

The functions are not only dispatchers. They also enforce mode-specific constraints. The no-deposit creation function rejects any initial value, the buyer-sponsor creation function requires the caller to be the buyer, and the vendor-sponsor creation function requires the caller to be the vendor. These checks make the selected mode part of the deployed contract's on-chain history.

In the application deployment helper, the factory is used when clone-based deployment is enabled, a factory address is present, and the selected mode is a non-hardcoded specialized variant. The normal interactive path is kept as a direct account deployment because it preserves the existing ERC-4337-compatible normal contract. The clone factory is nevertheless available for normal-mode measurements and for comparing direct deployments with clone-based deployments.

This distinction is important for gas accounting. The factory and implementation contracts are infrastructure costs: they are deployed once and can then be reused. The marginal cost of a new clone-based exchange is the cost of creating and initializing the minimal proxy, not the cost of deploying the full implementation again. This is why later experimental tables separate one-time infrastructure gas from per-exchange gas.

6.4 Specialized optimistic escrow contracts

The optimistic contract family has two forms: direct contracts and clone contracts. They share the same high-level state machine, but they are used in different integration contexts.

The direct variants extend a common direct base. They are full contract deployments with constructor arguments. They are useful as fallbacks and are used by the hardcoded `SHA256` path. The direct normal contract is also the path that keeps the ERC-4337-style account functionality inherited from the original application.

The clone variants extend `OptimisticSOXAccountCloneBase.sol`. They are initialized after creation by the factory. The clone base removes the ERC-4337 account logic and keeps only the optimistic escrow behavior needed by the specialized variants. This was a deliberate simplification: the variants are measured and demonstrated mainly with direct transactions, while the normal path remains available when the existing account-abstraction flow is needed.

The four clone implementations encode the deposit differences described in Chapter 5. The normal clone requires the sponsor fee and waits for buyer payment. The no-deposit clone requires zero value and makes the buyer payment equal to the agreed price only. The $S = B$ clone requires the buyer to be the deployer, records the buyer payment during initialization, and starts directly in the key-waiting state. The $S = V$ clone requires the vendor to be the deployer, records the sponsor deposit, and still waits for buyer payment.

Both direct and clone bases contain the same dispute-sponsor entry points needed after the honest path fails. In particular, they include:

- a self-sponsored buyer dispute deposit path;
- an authorization-based external buyer dispute sponsor path;
- a self-sponsored vendor dispute deposit path;
- a runtime choice between normal and self-sponsored dispute deployment.

The authorization-based buyer path is especially important. It prevents an external S_B from unilaterally claiming that the buyer is unhappy. The

signed message includes the chain id, the optimistic contract address, the buyer, the proposed buyer-side dispute sponsor, and the commitment. This binds the authorization to one exchange and one proposed sponsor.

6.5 Specialized dispute game contracts

The dispute architecture is split into deployer libraries and dispute account contracts. The deployer libraries are small selectors that instantiate the appropriate dispute account: normal, self-sponsored, or hardcoded SHA256. The optimistic account decides which deployer to use when the vendor-side dispute sponsor deposit is made.

The normal dispute account, `DisputeSOXAccountNormal.sol`, keeps the generic dispute behavior. It implements the challenge-response loop, Step 8 verification, and the normal Step 9 behavior. This path is used when the dispute sponsors are external or when the self-sponsored specialization is not applicable.

The self-sponsored dispute account, `DisputeSOXAccountSelfSponsored.sol`, inherits from the normal dispute account and overrides only the Step 9 handling. When both $S_B = B$ and $S_V = V$, it does not relaunch the challenge loop for a new sponsor configuration. Instead, it moves directly toward the terminal outcome determined by the losing party. This keeps the implementation small and makes the optimization auditable: the rest of the dispute logic is inherited unchanged, and the only semantic change is the Step 9 transition.

This inheritance structure was chosen to reduce implementation risk. The project did not rewrite the complete dispute verifier for the self-sponsored case. It specialized the part that is actually different. This also makes the gas measurements easier to interpret, because a saving in the self-sponsored case can be attributed to removing unnecessary iterations rather than to a hidden rewrite of the verifier.

6.6 Hardcoded SHA-256 dispute contracts

The hardcoded SHA256 path is implemented separately from the clone factory. Its optimistic contract is `OptimisticSOXAccountHardcodedSHA256.sol`. This contract stores three immutable pieces of metadata: the description hash, the plaintext length, and the ciphertext IV. At deployment time, it verifies that the number of ciphertext blocks and the number of circuit gates match the hardcoded SHA256 layout.

When a dispute is triggered from this contract, it uses the hardcoded SHA-256 dispute deployer, which instantiates the corresponding hardcoded dispute account. The dispute account overrides the generic Step 8 entry points so that the vendor no longer provides the circuit gate bytes as the

source of truth and no longer provides the π_1 Merkle proof for $h_{circuit}$. In the strict hardcoded tests, the corresponding inputs are empty.

The reconstruction logic is isolated in `HardcodedSha256CircuitLib.sol`. Given the gate number, the number of blocks, the description hash, the plaintext length, and the IV, the library computes the expected 64-byte encoding of the selected gate and checks its hash. It covers the AES-CTR input gates, the SHA-256 padding gates, the SHA-256 compression chain, the description constant, and the final comparison gate.

The hardcoded contract does not remove all witness data from Step 8. The smart contract does not store the whole ciphertext nor every intermediate value. The vendor still provides values needed to evaluate the selected gate, but those values are verified against the appropriate commitments and accumulator proofs before the gate result is accepted. The architectural change is therefore specific and limited: g_i is reconstructed by the contract, while v_i -style witness values remain externally supplied and authenticated.

This design also explains why hardcoded SHA256 is implemented as a dedicated path instead of a flag inside every dispute contract. The hardcoded path changes the Step 8 verifier. Carrying both generic and hardcoded verification in the same contract would put the project back into the monolithic pattern that the final architecture was meant to avoid.

6.7 Native CLI path for large files

The original application relied heavily on WebAssembly and browser-adjacent execution. That is convenient for small demonstrations, but it becomes fragile for very large files. The project therefore uses native Rust command-line tools for large-file experiments.

The main precontract tool is `src/wasm/src/bin/precontract_cli.rs`. It reads an input file, chooses or receives a 16-byte AES key, computes the V2 precontract values, writes the ciphertext and circuit bytes to disk, and returns a JSON object containing: `description_hex`, `h_ct_hex`, `h_circuit_hex`, `commitment_c_hex`, `commitment_o_hex`, `num_blocks`, `num_gates`, output paths, and the key.

The web API uses this CLI in multipart mode. Instead of loading the upload through a browser-only computation path, `/api/precontracts` streams the uploaded file to a temporary file and calls `precontract_cli`. In Electron mode, the application can run the same native precomputation locally and send the resulting JSON to the backend. In both cases, the backend stores the precontract metadata and moves the ciphertext artifact to the application upload directory.

The large-file dispute benchmark uses a second native tool, `src/wasm/src/bin/hardcoded_sha256_dispute_cli.rs`. This tool is intentionally narrower than a full generic dispute prover. It validates the number of

blocks and gates for the hardcoded SHA256 layout, computes the sequence of $hpre_i$ values for the left-branch benchmark path, and prepares the final left-boundary Step 8 witness. This is the tool used to make the 1 GiB hardcoded dispute measurements practical.

The distinction between the two native tools matters. The precontract CLI is part of the interactive creation path. The hardcoded dispute CLI is a benchmark and measurement tool for a selected dispute scenario. It does not mean that the full browser UI now supports every possible 1 GiB generic dispute interaction end-to-end.

6.8 Database and artifact management

The application stores precontract and exchange metadata in SQLite through `src/app/lib/sqlite.ts`. The schema is defined in `src/app/db/init.sql` and contains two main tables: `contracts` and `disputes`. The project added the `precontract_variant` column to `contracts`, with default value `normal`. The runtime initialization code also attempts to add this column to older local databases, which keeps existing development databases usable.

The `contracts` table stores the buyer and vendor addresses, price, tips, timeout, number of blocks, number of gates, commitment, opening value, protocol version, algorithm suite, acceptance status, sponsor address, deployed optimistic contract address, and session key information. The `disputes` table stores the dispute sponsors and dispute contract address for a contract id.

Encrypted files are stored outside the database. In web mode, the backend writes the upload to a temporary file, invokes the native precontract CLI, and then copies the produced ciphertext to the uploads directory under a deterministic name based on the contract id. In Electron mode, the precomputed ciphertext path is sent by the desktop shell and uploaded through the Electron bridge. This design avoids storing large ciphertexts directly in SQLite.

The backend also returns the key, $h_{circuit}$, and h_{ct} to the frontend after precontract creation. The current application stores some of these values in browser local storage for the demonstration workflow. This is acceptable for the local proof of concept, but it is not a production key-management design. The limitation is documented explicitly later in the report rather than hidden as an implementation detail.

6.9 Integration with the existing application

A central constraint of the project was to improve the implementation without breaking the existing proof-of-concept application. The final architecture

therefore integrates the new contracts through the existing application life-cycle:

1. the vendor creates a precontract and selects a transaction mode;
2. the backend computes or receives the Rust precontract output and stores metadata;
3. the buyer accepts the precontract and downloads the ciphertext;
4. the sponsor view deploys the optimistic contract using the selected mode;
5. the buyer pays if the selected mode still requires a separate payment;
6. the vendor releases the key and the exchange completes optimistically, or the parties move to the dispute path.

The deployment helper is the bridge between the application and the Solidity architecture. It reads deployed infrastructure addresses from the generated deployment JSON file, chooses the factory path when applicable, or falls back to direct deployment when the selected mode requires it. It also checks for unlinked library placeholders before deploying a direct contract, which prevents a class of confusing runtime failures where bytecode artifacts were not regenerated after a contract change.

The normal path remains compatible with the original ERC-4337 and EntryPoint demonstration. The specialized clone variants do not claim full account-abstraction support; their `supportsERC4337` function returns false through the clone base. This separation is intentional. It lets the project measure the gas benefit of narrower contracts without pretending that every optimized variant has the same transaction-sponsoring integration as the inherited normal account.

The final integrated architecture is therefore hybrid but controlled. The user-facing application can demonstrate normal sponsored exchanges and the main precontract variants. The benchmark suite can additionally exercise clone-based normal measurements, hardcoded SHA256 disputes, and large-file native paths. The report keeps these scopes separate in the experimental chapters so that an application demo, a gas benchmark, and a large-file native benchmark are not accidentally treated as the same result.

Chapter 7

Correctness and Security Analysis

7.1 Functional correctness of the optimistic path

This chapter analyzes implementation correctness and security at the level of the codebase developed during the project. It is not a formal proof of the SOX protocol. The goal is more practical: identify which properties are enforced by the implementation, which properties are covered by tests, and which issues remain as limitations or future work.

The optimistic path is implemented as a state machine. The relevant states are payment, key release, buyer-side dispute sponsor registration, vendor-side dispute sponsor registration, dispute, and final state. Each transition checks both the current state and the expected actor. In the direct normal ERC-4337-compatible contract, this check can be satisfied either by the actor directly or through a validated `UserOperation` context. In the specialized clone contracts, it is satisfied by direct calls from the expected address.

The precontract variants modify the optimistic path only where the protocol requires it. The normal path starts by waiting for buyer payment. The no-deposit path also waits for buyer payment, but removes the optimistic sponsor deposit and reduces the required buyer payment to the agreed price. The $S = B$ path fuses creation and buyer payment: the buyer is the deployer, the buyer deposit is recorded during initialization, and the contract starts waiting for the key. The $S = V$ path requires the vendor to be the sponsor, but still waits for the buyer payment.

Correctness of these transitions is checked at three levels. First, the frontend validates obvious role constraints before deployment. Second, the factory and specialized contracts repeat the same constraints on-chain. Third, the Hardhat tests verify the resulting state and deposits. In particular, the tests check that:

- the no-deposit variant records no sponsor deposit and rejects unexpected value;
- the $S = B$ variant starts after the buyer payment has already been recorded;
- the $S = V$ variant enforces the vendor as deployer and still waits for buyer payment;
- unsafe combinations, underfunded deployments, and unauthorized actor calls revert;
- the 12 role combinations studied in Phase 2 can be executed in the test matrix.

The application also contains a native precontract verification route. When the buyer views a non-accepted precontract, the UI can call `/api/precontracts/verify`, which invokes `check_precontract_cli`. This recomputes the ciphertext and circuit commitments from the stored ciphertext, description, commitment, and opening value. Thus, the implementation contains a concrete verification mechanism for the buyer-facing precontract data. The current proof-of-concept interface should still be read as a local demonstration client, not as a fully hardened wallet-integrated client.

7.2 Functional correctness of the dispute path

The dispute path begins only after the optimistic contract has reached the appropriate state. The dispute constructor checks that the optimistic contract is waiting for the vendor-side dispute sponsor, that a buyer-side dispute sponsor has already been set, that the circuit version is supported, that the number of gates is non-zero, and that sufficient funds are available. This prevents a dispute account from being initialized from an inconsistent optimistic state.

The normal dispute contract implements the binary-search structure of Step 7. The interval is represented by a , b , and `chall`. The buyer publishes a response for the current challenge, the vendor gives an opinion, and the interval is updated until a boundary or a single gate is reached. Depending on the final challenge value, the contract enters one of the Step 8 states: left boundary, right boundary, or ordinary gate verification.

The Step 8 verification path checks several commitments at once. For an ordinary gate, the contract opens the global commitment to recover $h_{circuit}$ and h_{ct} . It checks that the supplied gate belongs to the circuit accumulator, that ciphertext inputs used by the gate belong to h_{ct} , that previous values belong to the previous h_{pre} , and that the gate output extends

the accumulator to the current value. The Solidity implementation uses the V2 evaluator to recompute the selected gate result on-chain.

The Step 9 logic is also tested in both normal and self-sponsored configurations. In the normal dispute contract, Step 9 may relaunch the challenge loop when the losing party is not the corresponding sponsor. In the self-sponsored dispute contract, this loop is specialized: if $S_B = B$ and $S_V = V$, the contract moves directly toward completion or cancellation after the Step 8 result. This is intentionally a small override of Step 9 rather than a full rewrite of the dispute verifier.

Functional tests and performance tests both exercise the dispute path. The tests include normal disputes, self-sponsored disputes, hardcoded disputes, same-scope comparisons, and a native 1 GiB hardcoded benchmark that executes precontract generation, every challenge round, Step 8 left-boundary submission, and finalization. These tests do not constitute a formal proof, but they are strong regression tests for the implemented state transitions and the measured scenarios.

7.3 Correctness of the hardcoded SHA-256 mode

The hardcoded SHA256 mode has a narrower correctness objective than the generic circuit mode. It does not try to support arbitrary descriptions. It supports the case where the description is $desc = \text{SHA256}(x)$, and where the circuit layout is completely determined by public metadata.

The optimistic hardcoded contract fixes this mode at deployment time. It stores the description hash, plaintext length, and ciphertext IV, and verifies that the supplied number of blocks and number of gates match the hardcoded layout. This prevents the vendor from waiting until Step 8 to choose between the generic and hardcoded interpretations.

The dispute contract then reconstructs the expected gate bytes through `HardcodedSha256CircuitLib.sol`. The library determines whether the gate is an AES-CTR block gate, a padding gate, a SHA-256 compression gate, the description constant, or the final comparison gate. The strict hardcoded tests call Step 8 with empty generic circuit inputs: `gateBytes = 0x` and an empty π_1 . The contract therefore does not rely on a vendor-supplied circuit gate as the source of truth.

This does not mean that Step 8 becomes witness-free. The contract does not store the entire ciphertext or all intermediate values. The vendor still supplies values needed to evaluate the selected gate. These values are checked against h_{ct} , previous h_{pre} , and extension proofs before the result is accepted. The hardcoded improvement is therefore specific: the contract determines g_i , while the supplied v_i -style values remain authenticated witnesses.

The hardcoded mode also reduces exposure to the generic circuit-encoding ambiguities discussed below, because the gate bytes are reconstructed by the

contract for this particular SHA-256 layout. It does not, however, fix the generic V2 circuit format for other descriptions.

7.4 Sponsor safety and incentive preservation

The project changed the sponsor structure, so it was important not to introduce a path where one actor can force another actor into a disadvantageous state. The implementation addresses this through on-chain role checks and through explicit authorization when a third party acts as buyer-side dispute sponsor.

For precontract-time sponsor equalities, the checks are simple. The $S = B$ factory path and contract require the deployer to be the buyer. The $S = V$ path requires the deployer to be the vendor. The no-deposit path requires zero initial value and removes the sponsor reimbursement accounting from the optimistic phase. These checks are enforced on-chain, not only in the UI.

For dispute-time sponsorship, the delicate case is an external S_B . The supervisor correctly pointed out that an external buyer-side dispute sponsor must not be able to claim that the buyer is unhappy without the buyer's consent. The implementation therefore includes an authorization-based function for external S_B . The signed statement binds the buyer's authorization to the chain id, the optimistic contract address, the buyer address, the chosen S_B , and the commitment. This prevents reuse across different exchanges or different proposed sponsors.

For $S_B = B$ and $S_V = V$, the implementation uses explicit self-sponsored entry points. This makes the Step 4–5 fusion and the Step 9 specialization transparent. The gas benefit therefore comes from removing unnecessary repeated dispute iterations, not from skipping authorization checks or weakening the dispute verifier.

The economic incentive analysis remains within the scope of the implemented protocol model. The code preserves the relevant deposits and payouts of the tested scenarios, but it does not attempt to solve all off-chain availability or censorship risks. Those assumptions are separated below.

7.5 Encoding ambiguities in the inherited circuit format

The V2 circuit format encodes each gate on exactly 64 bytes:

$$\text{opcode} \parallel \text{sons} \parallel \text{parameters} \parallel \text{zero padding}.$$

Sons are signed 48-bit indices encoded on 6 bytes. Positive indices point to previous gates; negative indices point to ciphertext input blocks. The generator uses opcodes for AES-CTR, SHA2, CONST, XOR, and COMP.

The Solidity evaluator checks the opcode, parameter length, padding, arity-dependent constraints, and rejects zero gate indices in the extraction logic.

Nevertheless, the inherited format is not as clean as it should be for a final protocol specification. The arity is not explicitly encoded. For some calls, the Solidity parser infers arity from the position at which the remaining bytes become zero padding. For other Step 8 calls, the expected arity is effectively determined by the number of supplied witness values. This works for the generated circuits used in the tests, but it is not a robust canonical encoding rule.

The problematic cases are exactly the cases identified during the project discussion. A SHA2 gate may have one or two sons, and a CONST gate may have zero or one son. Without an explicit arity or distinct opcodes, a byte string can be interpreted differently depending on whether the parser treats a field as a son or as part of the constant payload. The current padding checks reduce accidental ambiguity, but they do not give the format the clean separation one would want in a paper-ready specification.

The clean fix is to define a new circuit version with unambiguous opcodes or an explicit arity byte. For example, one could separate CONST-without-son from CONST-with-son, and separate the first SHA-256 compression gate from subsequent SHA-256 compression gates. Such a change would require updating the Rust circuit generator, the Solidity evaluator, the hardcoded layout, tests, and all commitment computations. For this semester project, the format was analyzed and documented, but the generic V2 encoding was not replaced because that would have changed the baseline being measured.

7.6 Merkle tree domain separation issues

The second inherited cryptographic issue is the accumulator construction. The implementation builds Merkle-style accumulators with keccak256. Leaves are hashes of encoded objects, and internal nodes are hashes of two child hashes. In the current format, however, leaf hashing and node hashing are not domain-separated, and an odd element at a layer is copied upward unchanged.

This creates two well-known sources of ambiguity. First, copying a lonely node upward means that two different tree shapes can lead to the same value if the proof format does not fully determine the shape. Second, using the same hash domain for leaves and internal nodes allows a 64-byte encoded leaf to have the same byte length as two 32-byte child hashes concatenated. In a cryptographic specification, this is exactly the kind of ambiguity that should be avoided by construction rather than argued away case by case.

The source code itself already contains a warning about the lonely-node case in the Rust accumulator implementation. The Solidity verifier mirrors the same accumulator convention for compatibility with the generated proofs.

Therefore, this is not a bug introduced by the new sponsoring or hardcoded variants. It is an inherited accumulator-design limitation.

The appropriate fix is a new accumulator version with explicit domain separation. A clean design would use distinct prefixes, for example

$$\text{leaf} = H(0x00 \parallel \text{encodedLeaf}), \quad \text{node} = H(0x01 \parallel \text{left} \parallel \text{right}),$$

and a separate rule for single-child promotion, such as hashing it with its own prefix rather than copying it unchanged. This would need to be applied consistently to h_{circuit} , h_{ct} , h_{pre} , proof generation, and Solidity verification. It was not patched in the final implementation because it would create a new commitment format and make the gas comparison with the initial implementation less direct.

7.7 ERC-4337 trust and liveness assumptions

The original implementation uses ERC-4337-style UserOperations and an Alto bundler in the normal path. The contracts validate signatures through `validateUserOp`, store the role associated with the last validated operation, and then allow the account to execute the corresponding role action. This protects against arbitrary execution through the EntryPoint: the recovered signer must match the buyer, vendor, or dispute sponsor role expected by the state transition.

However, ERC-4337 also introduces liveness assumptions that are not part of the pure SOX protocol. A bundler may be unavailable, may refuse to include a UserOperation, or may require different gas policies. A paymaster or EntryPoint deposit can also become an operational dependency. These are not cryptographic breaks of the smart contract state machine, but they are relevant for a practical fair-exchange system.

For this reason, the final architecture separates the optimized clone variants from the normal account-abstraction path. The clone variants do not claim ERC-4337 support; they are used for direct-transaction gas measurements and for simplified modes. The normal path remains available for demonstrating the inherited account-abstraction integration. This separation avoids hiding bundler assumptions inside the gas-optimized variants.

7.8 Remaining security limitations

The implementation should be read as a research prototype, not as production software. The following limitations remain after the project:

- The generic V2 circuit encoding is not fully canonical because arity is not encoded explicitly for all gate families.

- The Merkle accumulators lack domain separation and use copy-up behavior for lonely nodes.
- The proof-of-concept UI uses predefined development addresses and does not implement a production wallet authentication model.
- Some cryptographic material and intermediate commitments are stored in browser local storage during the demonstration workflow.
- The backend stores ciphertext artifacts centrally. Commitments make tampering detectable, but the backend is still an availability dependency.
- The hardcoded mode is implemented only for the $desc = \text{SHA256}$ circuit layout, not for arbitrary descriptions.
- The 1 GiB hardcoded dispute benchmark demonstrates a specific full path and witness generation strategy; it is not a complete generic dispute UI for every possible large-file scenario.
- The work relies on extensive automated tests and measurements, but it does not include a machine-checked formal proof of the Solidity implementation.

These limitations do not invalidate the results of the project. They define the boundary of the contribution. The implemented work improves the gas architecture, adds and tests the main protocol variants, makes large-file experiments practical, and identifies the inherited cryptographic encoding issues that should be addressed in a future protocol version.

Chapter 8

Experimental Results

8.1 Measurement methodology and scopes

This chapter reports the gas and runtime measurements obtained during the experimental phase of the project. The main difficulty was not only measuring gas, but measuring the same quantity across different architectures. The implementation went through three relevant stages:

- the initial implementation, used as the baseline from the previous semester project;
- the intermediate monolithic implementation, where several variants were first added to the existing contracts as flags;
- the final specialized implementation, where the variants are split into dedicated contracts and, for the optimistic variants, into reusable implementations deployed through a factory.

All gas values in this chapter come from transaction receipts in the local Hardhat test environment. They should therefore be read as reproducible EVM gas measurements for the implemented contracts, not as predictions of end-user latency or fee paid on a public network. Off-chain timings were measured with the native Rust command-line tools used by the benchmarks. They are machine-dependent, but they are useful for comparing the relative cost of preparing precontracts and dispute witnesses.

The measurements were obtained from the Phase 4 benchmark suite, in particular the tests comparing the initial implementation, the monolithic implementation, the specialized final architecture, and the native 1 GiB hardcoded dispute path. The benchmark outputs were then reorganized by scope before being reported here, so that totals can be recomposed from the individual lines when needed.

The following scopes are used throughout the chapter.

Scope	Meaning
Fixed infrastructure	Gas paid once to deploy reusable components such as <code>SOXFactory</code> and optimistic implementation contracts.
Marginal optimistic cost	Gas paid for one new exchange after the reusable infrastructure already exists. For clone-based modes, this includes clone creation and the honest-path calls until completion.
Optimistic path	Gas needed to deploy or create the optimistic account and execute the honest exchange until the terminal complete state.
Dispute after trigger	Gas paid after the dispute account has been deployed, namely the challenge rounds, Step 8 submission, and finalization.
Full dispute path	Gas including the optimistic setup, dispute trigger/deployment, challenge rounds, Step 8, and finalization.
Off-chain runtime	Wall-clock time for native computation of precontract artifacts or dispute witnesses. It is not on-chain gas.

Table 8.1: Measurement scopes used in the experimental evaluation.

This separation is essential. For example, a factory-based clone has a very low marginal cost per exchange, but only after the factory and implementations have already been deployed. Conversely, a direct hardcoded deployment can be more expensive in the optimistic phase while still being useful in dispute, because it reduces the dispute deployer and the generic circuit verification work.

8.2 Baseline measurements of the initial implementation

The first baseline is the honest optimistic exchange in the initial implementation. In that architecture, every exchange deploys a full optimistic account contract. The complete optimistic path measured in the initial implementation costs 2,300,201 gas: 2,077,362 gas for deployment and 222,839 gas for execution.

Optimistic path	Initial	Interm. mono.	Difference
Deployment	2,077,362	2,813,862	+736,500
Execution until complete	222,839	232,920	+10,081
Total	2,300,201	3,046,782	+746,581

Table 8.2: Honest optimistic path before and after the first monolithic variant changes.

Table 8.2 shows the negative result that motivated the final architecture. The monolithic implementation was functionally useful, but it made the optimistic path 32.46% more expensive than the initial implementation. Most of the increase came from deployment, not from the honest-path calls. This confirmed that putting all variants into the same deployed bytecode was the wrong optimization strategy.

The same effect appears in the dispute initialization measurements on a 4 MiB file. The monolithic implementation is again more expensive than the initial implementation when the same scope is measured.

The baseline conclusion is therefore clear: the first implementation of variants did not optimize gas. It was an intermediate engineering step that made the variants executable and measurable, but its gas cost justified the move to specialized contracts.

8.3 One-time infrastructure deployment costs

The final optimistic architecture uses a reusable factory and reusable implementation contracts. These costs are paid once at protocol setup time. They

Dispute-related measure, 4 MiB	Initial	Interm. mono.	Difference
Dispute deployment / trigger	4,651,201	5,779,296	+1,128,095
Exchange + dispute triggering	4,977,003	6,058,222	+1,081,219
Optimistic deploy + dispute init	7,054,365	8,872,096	+1,817,731
total			
One challenge round average	82,624	103,282	+20,658
submitCommitment, SHA-256 gate	320,000	379,686	+59,686

Table 8.3: Initial implementation versus intermediate monolithic implementation on the same 4 MiB dispute scope.

are not paid again for each exchange.

Component	Deployment gas
SOXFactory	561,126
Normal optimistic clone implementation	1,492,453
no_S_deposit implementation	1,482,093
$S = B$ implementation	1,529,267
$S = V$ implementation	1,519,062
Total optimistic clone infrastructure	6,584,001

Table 8.4: One-time infrastructure cost of the clone-based optimistic architecture.

This table should not be added to every exchange. It is an amortized infrastructure cost. For a single isolated exchange, deploying this infrastructure would be expensive. For a system handling several exchanges, however, the marginal cost becomes the relevant measure. Compared with the initial optimistic total of 2,300,201 gas per exchange, the clone infrastructure is amortized after roughly four exchanges in the measured specialized optimistic modes.

8.4 Optimistic path gas costs

The final implementation provides two kinds of specialized optimistic measurements. The first kind uses direct specialized contracts. These already reduce the cost compared with the initial implementation, because each contract removes unrelated branches. The second kind uses minimal clones through **SOXFactory**. These are the lowest marginal-cost measurements, because the full implementation bytecode has already been deployed once.

The clone-based **no_S_deposit** mode is the cheapest measured honest exchange: 500,720 gas once the infrastructure exists. The $S = B$ and $S = V$ cases are slightly more expensive, but they remain below 560,000 gas. These

Specialized optimistic mode	Direct total	Clone marginal total	Clone gain vs initial
<code>no_S_deposit</code>	1,724,251	500,720	−78.23%
$S = B$	1,751,300	516,349	−77.55%
$S = V$	1,787,289	556,309	−75.81%

Table 8.5: Optimistic-path gas costs for the final specialized modes.

values are the main positive result on the optimistic path. The project reduced the per-exchange optimistic cost from roughly 2.3 million gas in the initial implementation to roughly 0.5–0.56 million gas in the final specialized clone architecture.

It is important not to compare these clone marginal values directly with a first deployment of the whole protocol. The fair comparison is either direct-specialized versus initial single-exchange deployment, or clone marginal versus initial per-exchange cost after the clone infrastructure has been paid once. Appendix A gives an explicit recomposition table for these final factory/clone costs.

8.5 Dispute path gas costs

For the dispute path, the main same-scope comparison uses a 4 MiB file and measures “optimistic deploy + dispute init total”. This scope combines the cost of creating the optimistic exchange with the cost of reaching the dispute initialization point. It is useful because it exposes both the deployment effect and the dispute trigger effect.

Architecture, same 4 MiB scope	Gas
Initial implementation	7,054,365
Intermediate monolithic implementation	8,872,096
Final specialized implementation	5,495,184

Table 8.6: Same-scope 4 MiB dispute initialization comparison.

The final specialized implementation reduces this scope by 38.06% compared with the monolithic implementation and by 22.10% compared with the initial implementation. The main reason is architectural: the optimistic deployment component becomes a clone creation, while the generic Step 8 verifier remains close to the monolithic cost in the normal non-hardcoded case.

A second same-scope comparison uses 16 KiB disputes and compares the intermediate monolithic diagnostic against the final specialized contracts for the four relevant dispute variants. This scope starts at the buyer-side

dispute sponsor step and includes the trigger, challenge rounds, Step 8, and finalization.

Dispute case, 16 KiB	Interm. mono.	Final specialized	Gain
Normal, external sponsors	8,345,424	7,432,647	−10.94%
Self-sponsored $S_B = B, S_V = V$	7,165,928	5,946,445	−17.02%
Hardcoded SHA256	8,310,230	5,195,022	−37.49%
Self-sponsored + hardcoded	7,193,323	4,042,185	−43.81%

Table 8.7: Same-scope 16 KiB dispute comparison between intermediate monolithic and final specialized contracts.

Table 8.7 is the clearest dispute-side summary of the final architecture. The normal case improves moderately because the verifier is still generic. The hardcoded cases improve much more because the dispute deployer and Step 8 verifier are specialized. The self-sponsored hardcoded case is the best measured 16 KiB dispute case in this scope, with a total of 4,042,185 gas.

8.6 Hardcoded SHA-256 gas impact

The hardcoded SHA256 mode has a specific purpose. It does not optimize the honest optimistic path directly. It optimizes the dispute path for the common description $desc = \text{SHA256}(x)$, by making the smart contract reconstruct the selected gate from public metadata instead of receiving the generic circuit gate and the Merkle proof π_1 for $h_{circuit}$.

The local Step 8 measurement confirms that π_1 is removed. For a 16 KiB circuit, the ordinary `submitCommitment` call costs 463,532 gas and carries 10 π_1 items. The hardcoded call costs 400,990 gas and carries no π_1 item, saving 62,542 gas on that local Step 8 call.

Size	Normal Step 8	Hardcoded Step 8	π_1 items	Saving
13 bytes	251,444	234,626	3 to 0	16,818
16 KiB	463,532	400,990	10 to 0	62,542

Table 8.8: Local Step 8 effect of removing the generic $h_{circuit}$ proof.

For large circuits, isolating only the $h_{circuit}/\pi_1$ membership check gives a larger local saving. At 1 GiB, the generic membership verification costs approximately 198,585 gas, while the hardcoded reconstruction costs between 29,639 and 32,390 gas depending on the selected gate. This is a local saving of about 166,000–169,000 gas. In a full dispute, this saving is diluted by the other costs: dispute deployment, challenge rounds, AES/SHA gate evaluation, h_{ct} proofs, h_{pre} proofs, and finalization.

Equivalent circuit size	Generic $h_{circuit}$ check	Hardcoded check	Local saving
900 MiB	192,267	29,639–32,390	159,877–162,628
1 GiB	198,585	29,639–32,390	166,195–168,946

Table 8.9: Isolated large-circuit effect of replacing π_1 by hardcoded reconstruction.

The largest full-path gain of the hardcoded work comes from the specialized hardcoded dispute deployer. In the intermediate monolithic hardcoded path, triggering a hardcoded dispute still paid for large generic deployment logic. In the final specialized hardcoded path, the trigger/deploy component decreases from 5,767,448 gas to 2,699,433 gas in the measured deployment-and-trigger scope. This explains why the final hardcoded dispute numbers improve far more than the isolated π_1 saving alone. A full component-level mirror of the monolithic hardcoded path is given in Appendix A.

8.7 Self-sponsoring gas impact

Self-sponsoring affects the dispute path differently from hardcoded SHA256. It does not change the circuit verifier. Instead, it changes the number of times the challenge loop must be executed. In the normal external-sponsor case, Step 9 may require a new loop with another sponsor configuration. In the $S_B = B, S_V = V$ case, this relaunch is unnecessary, because the disputing parties are already their own dispute sponsors.

The 16 KiB monolithic diagnostic makes this effect visible if one removes the fixed trigger and setup components and looks at the variable dispute part after the trigger. The normal external-sponsor case costs 2,450,528 gas after the trigger. The self-sponsored case costs 1,277,852 gas after the trigger, a reduction of 47.85%. The self-sponsored hardcoded case costs 1,215,298 gas after the trigger, a reduction of 50.41% against the normal variable part.

This is consistent with the protocol expectation: the gas reduction is not a tiny local saving, but comes from removing a repeated Step 7–8 loop. In full totals, the percentage is smaller because fixed costs such as dispute deployment remain present. This distinction between full totals and variable loop cost was one of the main lessons of the measurement phase.

8.8 Large-file measurements up to 1 GiB

The implementation was also tested on a 1 GiB file using the native Rust path and the final hardcoded SHA256 dispute. This is the most important scalability measurement of the project: it demonstrates that the implementation can generate the required off-chain artifacts and execute the complete

on-chain dispute path up to the terminal state.

Final hardcoded strict path	16 KiB	1 GiB
Optimistic before trigger	2,938,363	2,938,363
Trigger dispute / deploy dispute	2,721,171	2,721,171
Total optimistic + trigger	5,659,534	5,659,534
Challenge responses total	608,190	1,581,282
Opinions total	421,615	1,095,903
Step 8 <code>submitCommitmentLeft</code>	5,449,017	5,577,706
Finalization	75,667	75,667
Dispute after trigger	6,554,489	8,330,558
Complete measured path	12,214,023	13,990,092

Table 8.10: Complete final hardcoded strict path for 16 KiB and 1 GiB.

The 1 GiB file has 16,777,216 ciphertext blocks and 33,554,437 gates in the hardcoded layout. The binary search therefore uses 26 challenge rounds. In the measured self-sponsored hardcoded scenario, the complete path costs 13,990,092 gas, including the optimistic path, the dispute trigger, all challenge rounds, Step 8, and finalization.

A separate 1 GiB hardcoded benchmark with external sponsors and three Step 7–8 iterations was also run. The dispute after trigger costs 23,865,946 gas. Including the trigger gives 26,585,279 gas, and including the full optimistic path before the dispute gives 29,532,277 gas. This is not a simple multiplication by three of the one-iteration cost, because later iterations rewrite some storage slots that have already been initialized.

8.9 Off-chain runtime measurements

The 1 GiB benchmark uses the native Rust CLI, not the browser/WASM path. This distinction is important. The WASM path remains useful for integration and small or medium files, but the native CLI is the reliable path for large-file measurements because it avoids the memory pressure of JavaScript and browser-adjacent execution.

The ranges in Table 8.11 reflect repeated runs on the same machine. The gas values are stable transaction-receipt measurements, but wall-clock times depend on cache state and local I/O. The first $hpre_i$ computations are slower than the later ones because the benchmark forces the binary search to the left. The challenge index is approximately divided by two at each round, so the native code recomputes a smaller prefix/accumulator for later rounds. This affects CPU time only. It is not a gas effect.

The on-chain gas of the challenge rounds does not follow this division by two. For the 1 GiB benchmark, `respondChallenge` costs 1,581,282 gas over 26 rounds, or about 60,819 gas per round. `giveOpinion` costs 1,095,903

1 GiB runtime measure	Time	Interpretation
Precontract CLI	22.42–48.70 s	Mainly vendor-side preparation: read x , encrypt, compute commitments and metadata.
Native dispute generation	16.43–35.46 s	Aggregate benchmark time to prepare dispute data; not attributable to one protocol actor.
Total $hpre_i$ generation	9.47–11.92 s	Buyer-side generation of the published challenge responses over 26 rounds.
Local Hardhat transaction execution	0.30–0.56 s	Local wall-clock time to send the benchmark transactions; not a cryptographic runtime.

Table 8.11: Off-chain runtime measurements for the 1 GiB hardcoded benchmark.

Round	Challenge index i	$hpre_i$ time
0	16,777,219	4.58–6.04 s
1	8,388,610	2.23–2.46 s
2	4,194,305	1.17–1.33 s
3	2,097,153	0.60–0.71 s
4	1,048,577	0.31–0.39 s

Table 8.12: First $hpre_i$ generation times in the 1 GiB forced-left benchmark.

gas over 26 rounds, or about 42,150 gas per round.

8.10 Comparison summary

The final comparison is summarized in Table 8.13. The most important result is that the final architecture does not merely recover from the monolithic overhead; it improves over the initial implementation on the main same-scope measurements that were targeted by the project.

Result	Interpretation
Optimistic clone modes: 500,720–556,309 gas	Per-exchange optimistic marginal cost is reduced by about 75.8%–78.2% compared with the initial 2,300,201 gas optimistic path.
4 MiB specialized dispute-init scope: 5,495,184 gas	The final specialized architecture is 38.06% cheaper than the intermediate monolithic version and 22.10% cheaper than the initial implementation on the same scope.
16 KiB specialized dispute cases	All four measured variants improve over the intermediate monolithic equivalents; the best case is self-sponsored + hardcoded at 4,042,185 gas.
Self-sponsoring	The variable dispute loop cost drops by roughly half when Step 9 no longer relaunches unnecessary Step 7–8 iterations.
Hardcoded SHA256	The contract reconstructs g_i , removes π_1 , and reduces both local Step 8 gas and the specialized hardcoded dispute deployment path.
1 GiB hardcoded benchmark	A complete hardcoded dispute was executed up to End , with 26 challenge rounds and a full measured cost of 13,990,092 gas in the self-sponsored hardcoded scenario.

Table 8.13: Summary of the main experimental results.

8.11 Interpretation of results

The experimental results support three conclusions.

First, specialization is necessary. The monolithic implementation made the variants available, but it increased deployment gas and made several tables hard to interpret. The final split architecture is measurably better because mutually exclusive logic is no longer deployed together.

Second, the optimistic path is where the factory and clone architecture is most effective. The reduction from about 2.3 million gas to about 0.5 million gas per exchange is the largest clean improvement of the project. It directly addresses the cost of the honest case, which is the path expected to be used most often.

Third, dispute optimization has two different components. Self-sponsoring reduces the number of dispute iterations, while hardcoded `SHA256` reduces the generic circuit-verification burden and the hardcoded dispute deployment path. These optimizations are complementary: the strongest 16 KiB dispute result is obtained when both are enabled.

The remaining dominant costs are `triggerDispute`, deployment of dispute accounts, and the Step 8 evaluation of expensive gates, especially AES-related checks and the associated commitment proofs. These are the natural next targets for a future version of the implementation.

Chapter 9

Discussion

9.1 What actually reduced gas

The experimental results show that the main gas reduction did not come from a single micro-optimization. It came from separating concerns that had been merged in the first implementation of the variants. The intermediate monolithic contracts were useful because they made all variants executable, but they also deployed mutually exclusive logic together. The final implementation reduced gas by moving from this “one contract with many flags” model to specialized contracts whose bytecode matches the selected protocol mode.

The clearest example is the optimistic path. In the initial implementation, each exchange deployed a full optimistic account. In the final clone-based architecture, the expensive implementation contracts are deployed once and each new exchange only creates a minimal clone. This changes the gas model from a repeated full deployment to a fixed infrastructure cost plus a low marginal cost. As shown in Chapter 8, the measured marginal optimistic costs are 500,720 gas for `no_S_deposit`, 516,349 gas for $S = B$, and 556,309 gas for $S = V$. These are the strongest honest-path gains of the project.

The second real reduction comes from removing protocol steps that are unnecessary in some role configurations. The $S = B$ case fuses the sponsor deployment/payment role with the buyer role in the optimistic phase. The $S_B = B, S_V = V$ dispute case removes the need to repeat the challenge loop through Step 9 with another sponsor configuration. This is not a low-level Solidity trick. It is a protocol-level simplification reflected in the implementation. The measured variable dispute cost after the trigger is almost halved in the self-sponsored case, which matches the expected reduction from avoiding repeated Step 7–8 iterations.

The third reduction comes from the hardcoded SHA256 dispute path. Here, the contract does not accept a generic circuit gate and a proof π_1 as the source of truth. Instead, it reconstructs the expected gate from the

public metadata of the hardcoded $desc = \text{SHA256}(x)$ circuit. This removes the generic $h_{circuit}$ membership proof from Step 8. Locally, this saves 62,542 gas on the 16 KiB Step 8 measurement and about 166,000–169,000 gas on the isolated 1 GiB $h_{circuit}$ check. In the final specialized implementation, the hardcoded path also reduces the dispute deployer cost, which is why the full hardcoded 16 KiB dispute improves much more than the isolated π_1 saving alone.

The main lesson is that the successful optimizations are structural. They either avoid deploying unused bytecode, avoid executing unnecessary protocol loops, or remove a generic proof obligation when the circuit is already determined by the precontract.

9.2 What did not reduce gas as expected

Several ideas were useful conceptually but did not immediately reduce gas in the first implementation. The most important example is the intermediate monolithic contract. Adding `no_S_deposit`, $S = B$, $S = V$, self-sponsored disputes, and hardcoded `SHA256` as flags inside the existing contracts made the behavior configurable, but it increased deployment gas. The honest optimistic path rose from 2,300,201 gas in the initial implementation to 3,046,782 gas in the intermediate monolithic implementation. This result was not a measurement error: it was a consequence of deploying more code per exchange.

The first hardcoded measurements also looked weaker than expected when they were read as full-dispute totals. This was because the hardcoded change affects a specific part of Step 8, while a complete dispute also includes dispute deployment, challenge rounds, Merkle proofs for h_{ct} and h_{pre} , gate evaluation, and finalization. If one measures only the full total, the removal of π_1 is diluted by the remaining costs. The local hardcoded saving exists, but it is not the only component of the full dispute. This explains why the hardcoded optimization must be reported both locally and end-to-end.

Another important point is that the generic Step 8 verifier was not fundamentally optimized in the normal non-hardcoded path. In the 4 MiB same-scope comparison, the final specialized architecture reduces the overall dispute initialization cost because the deployment architecture is better, but the generic `submitCommitment` cost remains close to the monolithic implementation. Therefore, the final implementation improves the architecture around the verifier more than it improves the generic verifier itself.

Finally, ERC-4337 support should not be described as a gas optimization in this project. The inherited account-abstraction path is useful to demonstrate sponsored interactions and `UserOperations`, but it introduces operational components such as the `bundler` and `EntryPoint`. The specialized clone measurements deliberately use a simpler direct-call path. This sepa-

ration avoids claiming that account abstraction itself reduces the protocol gas. In this implementation, the gas reductions come from specialization and clone deployment, not from ERC-4337.

9.3 Cost relocation versus true optimization

One subtle point in the measurements is the difference between relocating a cost and removing a cost. The clone architecture relocates part of the optimistic deployment cost from every exchange to a one-time infrastructure deployment. This is a real practical gain when multiple exchanges share the same infrastructure, but it is not the same as making the first deployment free. The factory and implementation contracts cost 6,584,001 gas in total. That cost must be paid once before the marginal per-exchange savings become available.

This is still meaningful for a protocol implementation. In the initial architecture, the same optimistic bytecode is redeployed for every exchange. In the final architecture, the implementation bytecode is deployed once and reused. The marginal cost is the relevant quantity for a deployed service, while the fixed infrastructure cost is relevant for the first setup. The report therefore separates these two costs instead of adding the factory deployment to every exchange or ignoring it completely.

The hardcoded SHA256 mode is different. It does not merely move a cost from on-chain to an earlier deployment step. It removes the need to verify π_1 for $h_{circuit}$ in the hardcoded case, because the contract can reconstruct the expected circuit gate. However, it does not remove the need for witnesses altogether. The contract still needs authenticated values for ciphertext blocks and intermediate accumulators. These values are provided externally and verified against h_{ct} or h_{pre} . Thus, hardcoding removes a specific generic proof obligation; it does not make Step 8 free.

The off-chain large-file work is another example. Moving precontract generation and dispute witness computation to the native Rust CLI is not an on-chain gas optimization. It is a practical scalability improvement. It makes 1 GiB experiments possible without relying on a browser or a JavaScript/WASM memory model. The on-chain contract still verifies the submitted commitments and witnesses; the CLI only makes it feasible to generate the data needed by the protocol.

These distinctions matter because a fair comparison must compare the same scope. A result can be correct and still misleading if it mixes a fixed deployment cost with a marginal per-exchange cost, or a local Step 8 saving with a complete dispute total. The final experimental chapter was structured around this lesson.

9.4 Trade-offs of specialization

Specialization reduces gas, but it also introduces engineering trade-offs. The final architecture contains more contracts than the initial implementation: factory, clone implementations, direct variants, normal dispute accounts, self-sponsored dispute accounts, hardcoded dispute accounts, deployer libraries, and verifier libraries. This makes the bytecode paid by each exchange smaller, but it makes the source tree larger and increases the number of paths that must be tested.

This trade-off is acceptable in this project because the variants are protocol modes, not arbitrary runtime options. The mode `no_S_deposit`, $S = B$, and $S = V$ is known at precontract time. The hardcoded SHA256 mode is also fixed before dispute. Therefore, there is no reason to deploy a contract that contains all possible branches for one exchange. Specialized contracts match the protocol structure more closely than a single universal account.

The trade-off is more delicate for modes that are only known after Step 4, such as $S_B = B$ and $S_V = V$. These cannot be chosen at the initial precontract deployment in the general case. The implementation therefore specializes them at dispute deployment time. This is why the optimistic contract keeps enough logic to select the correct dispute deployer when the dispute sponsor roles are known. This design follows the protocol timing: precontract-time facts are specialized early, dispute-time facts are specialized later.

There is also a compatibility trade-off. The normal direct path preserves the inherited ERC-4337-compatible behavior. The clone variants are optimized for direct-call gas measurements and simplified execution. This means the final implementation does not claim that every optimized variant has the same account-abstraction integration as the normal path. This is a deliberate boundary: preserving all ERC-4337 logic inside every optimized clone would reintroduce the monolithic overhead that the project was trying to remove.

The strongest argument in favor of specialization is that it also helped with mainnet code size constraints. The original generic dispute contract and deployer were close to or above the EVM bytecode size limit in some configurations. Splitting the hardcoded dispute account, deployer, and circuit library produced components below the limit. This is an architectural benefit as well as a gas benefit.

9.5 Practicality of large-file fair exchange

The project made large-file experiments concrete. The native 1 GiB benchmark executed a complete hardcoded dispute path up to the terminal state. This is important because the previous browser-oriented workflow was not

a reliable way to handle such files. The final benchmark shows that the protocol implementation can generate large precontract artifacts, compute dispute witnesses, execute 26 challenge rounds, perform Step 8, and finalize.

The result should nevertheless be interpreted carefully. The 1 GiB measurement is a controlled benchmark, not a production user experience. The file is processed by native Rust tooling, the blockchain is a local Hardhat network, and the measured transaction time does not include public-network latency or mempool behavior. The benchmark proves feasibility of the implemented cryptographic and on-chain path; it does not prove that the current UI is ready for production-scale 1 GiB disputes.

The off-chain times are also significant. Preparing the 1 GiB precontract took 22.42–48.70 seconds across repeated runs on the benchmark machine. Generating all $hpre_i$ responses for the forced-left dispute took 9.47–11.92 seconds. These values are acceptable for an experimental proof of concept, but they are visible to users. They also show why the native CLI is necessary: the computation is not just a small frontend task.

On-chain, the 1 GiB self-sponsored hardcoded path costs 13,990,092 gas in the measured complete scenario. This is not negligible, but it remains a concrete executed path rather than a theoretical estimate. With external sponsors and three Step 7–8 iterations, the measured total including the optimistic path is 29,532,277 gas. This larger value shows why self-sponsoring and avoiding unnecessary iterations matter in practice.

The practical conclusion is therefore balanced. The implementation now handles 1 GiB experiments in a meaningful end-to-end way, but the current best path relies on the hardcoded SHA256 specialization and native tooling. A fully generic, browser-first, production-ready large-file fair exchange remains future work.

9.6 Lessons learned from the implementation

The first lesson is that implementation-level gas intuition is unreliable without same-scope measurements. Before the measurements were reorganized, several numbers looked contradictory because fixed infrastructure costs, marginal exchange costs, full dispute totals, and local Step 8 costs were mixed together. Once the scopes were separated, the results became coherent: monolithic flags increase deployment cost, clones reduce marginal honest-path cost, self-sponsoring reduces repeated loops, and hardcoding removes a specific generic proof.

The second lesson is that protocol variants should be aligned with when information becomes available. The modes $S = B$, $S = V$, and `no_S_deposit` are determined at precontract time, so they can be deployed as distinct optimistic contracts. The modes $S_B = B$ and $S_V = V$ are known only after the buyer expresses dissatisfaction and the dispute sponsors are selected, so

they belong in the dispute deployment decision. This timing distinction is important both for correctness and for gas optimization.

The third lesson is that security review can change the interpretation of performance work. The hardcoded **SHA256** path is not only a gas optimization; it also reduces reliance on the generic circuit encoding for that particular description. However, the review also identified inherited issues in the generic V2 circuit format and Merkle accumulator. These issues do not invalidate the measured gas improvements, but they show that a clean next version should update the commitment format, not only the gas architecture.

The fourth lesson is that an implementation can be improved without hiding its limitations. The final version is better than the initial implementation on the main same-scope measurements, and it supports large-file benchmarks that were not practical in the browser. At the same time, it is still a research prototype with local accounts, a local backend, development-chain measurements, and an inherited generic encoding that deserves a new version. Stating these boundaries explicitly makes the results more credible, not weaker.

Overall, the project moved the implementation from a functional but expensive one-contract-per-exchange prototype toward a more realistic architecture: reusable optimistic infrastructure, specialized modes, hardcoded dispute verification for **SHA256**, and native tooling for large files. The remaining work is now clearer: reduce the dominant dispute costs, clean the cryptographic encoding, and decide how far the factory/registry architecture should go in a future version.

Chapter 10

Limitations and Future Work

10.1 Current prototype limitations

The final implementation is a research prototype. It is substantially more complete and more measurable than the initial implementation, but it should not be presented as a production-ready fair-exchange system. The prototype runs on a local development stack, uses local accounts, stores artifacts through a local backend, and measures gas on a local Hardhat network. These choices are appropriate for a semester project focused on protocol variants and gas measurements, but they leave several engineering and security tasks open.

The first limitation is the user model. The application uses predefined development addresses and role selection through the local interface. It does not implement a production authentication flow where the vendor, buyer, sponsor, and dispute sponsors connect through real wallets and explicitly sign every role-dependent action. Some authorization checks are implemented on-chain, and the buyer-side dispute sponsor authorization is bound to the chain id and contract address. However, the surrounding user management remains a proof-of-concept environment.

The second limitation is the backend model. The implementation stores ciphertext artifacts, precontract data, and local metadata through the application backend and SQLite database. The on-chain commitments make tampering detectable, but the backend is still an availability dependency. If a production deployment wants to avoid this dependency, it would need a more robust artifact distribution layer, for example content-addressed storage, replication, or a clearly specified off-chain availability protocol.

The third limitation is the separation between the optimized paths and the inherited account-abstraction path. The normal direct contract keeps the ERC-4337-compatible behavior of the initial implementation. The clone-based optimized variants are measured through direct calls and are intentionally thinner. This is the right choice for gas optimization, but it means that the project does not yet provide one unified production interface

where all optimized variants are available through the same wallet and bundler flow.

The fourth limitation is proof coverage. The test suite covers the main targeted scenarios: normal optimistic execution, the three main optimistic variants, self-sponsored disputes, hardcoded **SHA256**, same-scope gas comparisons, and the 1 GiB hardcoded benchmark. This is strong evidence for the implemented paths, but it is not a formal proof of the Solidity implementation. A production version would require fuzzing, invariant tests, adversarial tests around malformed proofs, and preferably a formal specification of the state machines and commitment formats.

10.2 Verification workflow limitations

The implementation contains verification mechanisms, but they are not yet a complete production verification workflow. This distinction is important. The buyer-facing precontract modal exposes a “Verify commitment” action that calls `/api/precontracts/verify`. That route invokes the native `check_precontract_cli` binary and recomputes the public commitment data from the stored ciphertext, description, commitment, and opening value. Thus, the codebase does include a concrete verification path for the received precontract data.

However, this verification is not enforced as a backend precondition for acceptance. The acceptance route `/api/precontracts/accept` currently sets the contract as accepted from the submitted identifier. It does not require a previously stored successful verification result, nor does it bind the acceptance transaction to a verified h_{ct} and $h_{circuit}$ record. In the proof-of-concept UI, the buyer is expected to click the verification button before accepting. In a production version, acceptance should be blocked unless verification succeeds and the verified values are persisted and bound to the exchange.

The same limitation exists in the dispute workflow. The contracts verify the submitted commitments and witnesses on-chain during Step 8, and the native benchmarks generate the data needed for complete measured disputes. Nevertheless, the web API for proof generation is still partial: `/api/proofs/compute` currently supports only state 4 (`WaitVendorDataRight`). It is therefore not a complete production API for every possible Step 8 state. A full implementation should support all Step 8 cases used by the protocol and should make the actor-side local verification explicit before the actor signs the next dispute action.

This also affects runtime interpretation. The 1 GiB measurements include native generation of buyer-side h_{pre_i} values and the on-chain `giveOpinion` calls, but they do not isolate a complete production UI workflow in which V locally recomputes every buyer response before deciding what to sign. For

conservative protocol-level performance tables, such verification time should be counted for the verifying actor, even if the current benchmark does not expose it as a separate UI measurement.

10.3 Browser and WebAssembly limitations for large files

One concrete outcome of the project is that the browser/WASM path is not the right path for very large files. The WebAssembly integration remains useful for the interface and for small or medium examples, but the large-file benchmarks rely on the native Rust CLI. This is not accidental. A 1 GiB exchange requires reading large files, encrypting them, constructing commitments, and generating dispute witnesses. Doing this inside a browser or browser-adjacent JavaScript runtime creates avoidable memory pressure and makes results less reliable.

The measured 1 GiB path should therefore be described precisely: it is a native CLI benchmark integrated with the Solidity dispute contracts, not a complete browser-based 1 GiB user experience. This is still an important result, because it demonstrates that the protocol artifacts and the on-chain dispute can be produced and checked at this scale. It also clarifies the architecture that a future large-file application should use.

A natural future direction is to make the native path a first-class backend worker. Instead of asking the frontend to handle large cryptographic computations directly, the interface could delegate precontract generation, buyer-side $hpre_i$ computation, and dispute witness generation to a native service. The frontend would then orchestrate the exchange and display progress, while the native worker would handle file streaming and cryptographic artifact generation.

This future work should also include streaming and incremental verification. The current large-file benchmark proves feasibility, but it is still benchmark-oriented. A production implementation would need careful progress reporting, resumability, cancellation, disk space management, and checks that intermediate artifacts are not corrupted or mismatched with the on-chain commitments.

10.4 Toward a registry or factory-first architecture

The final implementation already introduces a factory for clone-based optimistic variants, but it does not fully redesign the protocol around a registry. Each exchange still gets a dedicated optimistic account or clone, and disputes still deploy dispute-specific accounts. This is a strong improvement over redeploying full optimistic bytecode every time, but it is not necessarily the end point of the architecture.

A more radical future architecture would use a factory-first or registry-first model. In such a design, reusable contracts would be deployed once, and each exchange would be represented by an `exchangeId` in a shared registry or by a minimal deterministic proxy. The objective would be to reduce repeated deployment costs even further and make Step 1+2 fusion easier to express. This would be a larger redesign than the one carried out in this project, because state layout, role authorization, dispute deployment, and artifact references would all need to be specified around the registry.

The main advantage would be gas efficiency and clearer lifecycle management. A registry can avoid deploying large bytecode for every exchange and can make it easier to reuse a sponsor approval if a buyer abandons the flow before paying. It could also centralize event emission, indexing, and off-chain discovery of exchanges.

The main risk is complexity. A shared registry must avoid cross-exchange state confusion, must isolate deposits correctly, and must keep the dispute logic auditable. The current clone architecture is a conservative intermediate step: it captures most of the deployment gain while keeping each exchange isolated. Moving to a registry should therefore be treated as a separate design task, not as a small patch.

10.5 Step 1+2 fusion for $S \neq B$

The project implemented the trivial Step 1+2 fusion for $S = B$: the buyer is also the initial sponsor, so deployment and buyer payment can be combined. A more difficult and interesting future direction is fusing Step 1+2 when $S \neq B$. The goal is to avoid a situation where S deploys or initializes a contract and then loses that deployment if B never pays.

One possible design is to move the loading of the precontract parameters from Step 1 to Step 2. The sponsor S would sign an approval for a specific set of precontract parameters. Then B would submit, in one transaction, the buyer payment, the precontract parameters, and the signature from S . If B does not submit this transaction, the sponsor has not locked a dedicated contract instance for that buyer. The same idea could be combined with deterministic factory deployment or with a registry entry created only when the buyer arrives with a valid approval.

This direction has to be studied carefully with ERC-4337. If a bundler or paymaster is expected to sponsor the transaction, it must be able to validate the `UserOperation` before execution. That means the signature from S , the expected buyer payment, the selected precontract parameters, and the target deterministic address or registry entry must all be checkable from the `UserOperation` data. If this validation requires executing state changes that are only available after deployment, the design becomes incompatible with a clean pre-sponsorship check.

For this reason, Step 1+2 fusion for $S \neq B$ is best treated as future work on the deployment architecture. It is closely connected to the registry/factory-first direction. The current implementation provides the measurements and specialized contracts needed to motivate this work, but it does not implement the full signed-precontract deployment protocol.

10.6 Cleaner circuit encoding

The inherited V2 circuit encoding is functional for the generated circuits used in the benchmarks, but it is not the cleanest format for a final protocol specification. Each gate is encoded on 64 bytes with an opcode, son indices, parameters, and zero padding. Some gate families have variable arity. In particular, SHA2 can use one or two inputs, and CONST can use zero or one input. In the current format, arity is not encoded as an explicit field or as a distinct opcode family.

This creates avoidable ambiguity at the specification level. Even if the current generated circuits and parser checks behave consistently for the tested paths, a clean format should not rely on inferring arity from padding conventions or from the number of witness values provided in Step 8. A paper-ready and audit-ready version should make the interpretation of every 64-byte gate canonical.

A future V3 circuit format should therefore introduce either explicit arity or distinct opcodes. For example, it could separate:

- the first SHA-256 compression gate from subsequent SHA-256 compression gates;
- CONST-without-input from CONST-with-input;
- any future gate family whose interpretation depends on optional sons.

This change would affect the Rust circuit generator, the Solidity evaluator, the hardcoded SHA256 reconstruction library, the test vectors, and the computation of $h_{circuit}$. It was intentionally not patched during the gas-optimization phase because it would have changed the commitment format and made comparison with the initial implementation less direct. It is nevertheless one of the most important future security cleanups.

10.7 Merkle domain separation

The accumulator construction used for $h_{circuit}$, h_{ct} , and h_{pre} should also be revised in a future version. The current implementation uses keccak256 for leaves and internal nodes without an explicit domain separator, and lonely nodes at an odd level can be copied upward. This convention is inherited

from the existing implementation and is mirrored by the Solidity verifier for compatibility with the generated proofs.

The limitation is structural. If leaves and internal nodes use the same hash domain, a 64-byte encoded leaf has the same input length as the concatenation of two 32-byte child hashes. This creates a classical ambiguity risk in Merkle-style constructions. Similarly, copying a lonely node upward can make different tree shapes share the same root unless the proof format completely rules out the ambiguity.

A cleaner accumulator version should domain-separate leaves and internal nodes. A simple candidate is:

$$\begin{aligned}\text{leaf}_i &= \text{keccak256}(0x00 \parallel \text{encodedLeaf}_i), \\ \text{node} &= \text{keccak256}(0x01 \parallel \text{left} \parallel \text{right}).\end{aligned}$$

It should also replace copy-up behavior with an explicit single-child rule, for example a separate prefix for promoted nodes. This change must be applied consistently to all accumulators and proofs. Like the circuit encoding change, it would create a new commitment version. It is therefore future work rather than a safe last-minute patch to the measured implementation.

10.8 Wallet integration

The current interface is a development interface. It allows the project to demonstrate the protocol flow, select roles, create precontracts, accept exchanges, and trigger contract actions. It does not yet provide a production wallet experience.

A production wallet integration should replace local role selection by explicit wallet authentication. Each actor should connect through a wallet, see the exact action being signed, and sign the relevant message or transaction. For off-chain authorizations, the signed payload should include the chain id, contract address or deterministic deployment parameters, exchange identifier, role, amount, deadline, and any commitment being approved. This would reduce ambiguity and make signatures auditable by the user.

Wallet integration also matters for the dispute path. During a dispute, the buyer and vendor must understand whether they are submitting a challenge response, giving an opinion, or providing Step 8 witnesses. A production interface should expose the protocol state in human-readable form and prevent users from signing stale or wrong-state actions.

This work is mostly engineering, but it is security-relevant. A fair-exchange protocol can be correct on-chain and still be unsafe in practice if users cannot reliably understand what they are authorizing. The current implementation is sufficient for experiments and demonstrations; a deployed system would require this layer.

10.9 Production deployment considerations

A production deployment would require several additional checks beyond the scope of the semester project. First, the contracts should be audited with the final commitment format fixed. The current code was tested extensively for the targeted scenarios, but an audit would need to review all state transitions, payout paths, timeout behavior, malformed proof inputs, and edge cases around deposits and sponsor roles.

Second, the deployment model should be finalized before optimization continues. The final project implementation already shows that deploying one large contract per exchange is not ideal. Future work should decide whether the long-term architecture is clone-based, registry-based, or a hybrid. That decision affects gas, upgrade strategy, indexing, precontract signatures, and ERC-4337 compatibility.

Third, the timeout and fee parameters should be calibrated for the target chain. Local Hardhat measurements do not capture public-network congestion, fee volatility, block time variance, or MEV/mempool behavior. A production system would need conservative timeouts and clear rules for who bears gas risk in the optimistic path and in dispute.

Fourth, artifact availability must be specified. The protocol can detect tampering through commitments, but it still needs the ciphertext and witness data to be available when an actor wants to verify or dispute an exchange. A production deployment should therefore define where artifacts are stored, how they are addressed, how they are replicated, and how users recover from backend unavailability.

Finally, the next gas targets are now well identified. The optimistic path has already been reduced substantially through clones. The remaining dominant on-chain costs are dispute deployment, `triggerDispute`, and Step 8 evaluation, especially for AES-related gates and the associated h_{ct}/h_{pre} proofs. Future optimization should focus on these components rather than reintroducing a monolithic contract with more flags.

In summary, the project reached the intended research-prototype milestone: it implemented and measured the main variants, corrected the architecture after the monolithic negative result, and demonstrated a 1 GiB hardcoded dispute path. The next version should not simply add more patches. It should clean the cryptographic formats, settle the deployment architecture, integrate real wallets, and then optimize the remaining dispute bottlenecks on top of that cleaner foundation.

Chapter 11

Installation, Execution, and Demonstration Guide

11.1 System requirements

The implementation was developed and tested in a Linux/WSL-style environment. The exact gas measurements reported in Chapter 8 were obtained on a local Hardhat network. Reproducing the project therefore requires both the web stack and the native cryptographic tooling.

The required tools are:

- Node.js, version 22 or later;
- npm, used by the Next.js frontend and the Hardhat project;
- Rust and Cargo, used to build the native precontract and dispute CLIs;
- `wasm-pack`, needed only when rebuilding the frontend WebAssembly bundle;
- `sqlite3`, used to initialize the local database;
- `pnpm` and Foundry tooling, used by the Alto ERC-4337 bundler setup;
- a Unix-compatible shell environment, preferably Linux or WSL.

The project contains an installation script that checks most of these dependencies and installs the JavaScript packages. The script is useful, but the manual commands below are also given so that the deployment can be reproduced and debugged step by step.

11.2 Repository setup

The repository should be cloned with its submodule, because the Alto bundler is stored in `bundler-alto` as a Git submodule.

```
git clone --recurse-submodules <repository-url>
cd sox_implementation-DOAB
git submodule update --init --recursive
```

From the repository root, the standard installation command is:

```
chmod +x install.sh
./install.sh
```

The script installs the root Next.js dependencies, the Electron dependencies, the Hardhat dependencies, initializes the SQLite database if needed, and sets up the Alto bundler. It also builds the native `precontract_cli` if Cargo is available. If the script is not used, the minimum manual installation is:

```
npm install
cd src/hardhat && npm install && cd ../..
cd desktop && npm install && cd ..
```

The local database is stored in `src/app/db/sox.sqlite`. To create a fresh database, remove the old file and replay the schema:

```
rm -f src/app/db/sox.sqlite
sqlite3 src/app/db/sox.sqlite < src/app/db/init.sql
```

This resets the local application state. It does not reset the blockchain node; restarting Hardhat is the clean way to reset the chain.

11.3 Building the Rust and WebAssembly components

The project uses Rust in two different ways. The frontend imports a WebAssembly bundle from `src/app/lib/crypto_lib`. The large-file benchmarks use native Rust binaries from `src/wasm/target/release`. These are separate build targets and should not be confused.

To rebuild all native CLI binaries used by the benchmarks, run:

```
cd src/wasm
cargo build --release --bins
cd ../..
```


The most important binaries are:

- `precontract_cli`, which generates precontract artifacts;
- `check_precontract_cli`, which verifies received precontract data;
- `compute_proofs_cli`, which supports the current proof-generation API;
- `hardcoded_sha256_dispute_cli`, which generates hardcoded dispute data for the large-file benchmark.

To rebuild the WebAssembly bundle used by the web application, install `wasm-pack` if necessary and run:

```
cd src/wasm
./deploy.sh
cd ../../
```

This writes the generated JavaScript and WASM files to `src/app/lib/crypto_lib`. The Hardhat performance tests that import the WASM bundle expect these files to exist.

11.4 Deploying the local smart contract stack

The local blockchain node must be running before contracts are deployed. In a first terminal, start Hardhat:

```
cd src/hardhat
npx hardhat node
```

In a second terminal, from the repository root, deploy the complete local contract stack:

```
./deploy-all.sh
```

The deployment script performs three main tasks. It deploys the local `EntryPoint`, deploys the simulation contracts needed by `Alto`, and deploys the `SOX` libraries, dispute deployers, clone implementations, and `factory`. The generated addresses are written to `deployed-contracts.json` and `src/deployed-contracts.json`. These files are used by the web application and the deployment helper.

If the Hardhat node is restarted, the deployed addresses are no longer valid. In that case, run `./deploy-all.sh` again before using the application or the bundler.

Terminal	Command
1	<code>cd src/hardhat && npx hardhat node</code>
2	<code>./deploy-all.sh</code> after Terminal 1 is ready
3	<code>./run-alto.sh</code> after deployment succeeds
4	<code>npm run dev</code> from the repository root
5, optional	<code>cd desktop && npm start</code> after Next.js is running

Table 11.1: Recommended local process layout.

11.5 Running Hardhat, Alto, and the frontend

The recommended development setup uses separate terminals. Keeping the processes separated makes errors easier to identify.

The web application is then available at <http://localhost:3000>. The local Hardhat RPC is available at <http://localhost:8545>. The Alto bundler RPC is available at <http://localhost:4337/rpc>.

The helper script `START_ALL.sh` can be used to coordinate the services, but the manual terminal layout in Table 11.1 is the most reproducible method for debugging and for demonstrations.

11.6 Running the optimistic demo

The simplest end-to-end demonstration is an honest optimistic exchange with the normal sponsored path. A 4 MiB test file can be created with:

```
mkdir -p demo_files
truncate -s 4M demo_files/demo_4MiB.bin
```

With Hardhat, the deployed contracts, Alto, and Next.js running, open the web interface at <http://localhost:3000>. The demo flow is:

1. Select the vendor role and create a new precontract.
2. Choose the buyer address, upload `demo_files/demo_4MiB.bin`, and keep the normal sponsored mode for the standard demo.
3. Submit the precontract. The backend calls the native precontract generator and stores the ciphertext and metadata.
4. Switch to the buyer role and open the non-accepted precontract.
5. Click “Verify commitment” to call `/api/precontracts/verify`. This checks the received ciphertext and commitment data with the native precontract checker.

6. Accept the precontract. The application marks it as accepted and downloads the ciphertext for the buyer.
7. Switch to the sponsor role and sponsor/deploy the optimistic contract.
8. Switch to the buyer role and send the buyer payment.
9. Switch to the vendor role and send the key.
10. The optimistic contract reaches the complete state if the exchange is honest.

For demonstrations focused on the new optimistic variants, the vendor can select SIMSOX (`no_S_deposit`), $S = B$, or $S = V$ in the precontract creation modal. The user interface enforces the obvious role constraints, and the corresponding on-chain path repeats the relevant checks.

11.7 Running gas benchmarks

The Hardhat test suite contains both correctness tests and performance tests. The following commands are the main tests used for the measurements in this report:

```
cd src/hardhat
npx hardhat test test/Phase3Variants.test.ts
npx hardhat test test/Phase3ExhaustiveAndGas.test.ts
npx hardhat test \
  test/performance/HanaComparisonCurrent.test.ts
npx hardhat test \
  test/performance/SpecializedContractArchitecture.test.ts
npx hardhat test \
  test/performance/SpecializedFinalSameScope.test.ts
```

The first two commands test the implemented variants and the monolithic variant matrix. The `HanaComparisonCurrent` benchmark remeasures the same scopes as the initial implementation. The specialized architecture benchmarks measure the factory, clone, self-sponsored, and hardcoded paths used in Chapter 8.

Gas reporter output can also be enabled through the environment variable already supported by the Hardhat configuration:

```
REPORT_GAS=true npx hardhat test
```

For stable comparisons, the repository should be measured on a fresh Hardhat state and with the same Solidity optimizer settings as the submitted implementation. The Hardhat configuration uses Solidity 0.8.28, optimizer enabled, `viaIR`, and optimizer `runs = 1`, because the project also targets bytecode-size reduction.

11.8 Running large-file benchmarks

The 1 GiB hardcoded benchmark requires the native Rust binaries. Build them first:

```
cd src/wasm
cargo build --release --bin precontract_cli \
  --bin hardcoded_sha256_dispute_cli
cd ../hardhat
```

Then run the benchmark:

```
npx hardhat test \
  test/performance/NativeHardcodedSHA256FullDispute1GB.test.ts
```

By default, this test uses a 1 GiB file. It creates the file in a temporary directory using file truncation, runs `precontract_cli`, runs the hardcoded dispute CLI, executes all challenge rounds, performs Step 8, and finalizes the dispute. The temporary directory is removed at the end of the test.

For a smoke test with the same code path but a smaller file, set the benchmark size environment variable. For example:

```
SOX_FULL_DISPUTE_SIZE_BYTES=16384 npx hardhat test \
  test/performance/NativeHardcodedSHA256FullDispute1GB.test.ts
```

The 1 GiB run requires enough disk space for the temporary plaintext, ciphertext, and intermediate artifacts, and it takes significantly longer than the small-file tests. It is therefore useful to run the small smoke test first when checking a new machine.

11.9 Troubleshooting

Most setup problems come from stale local state. A clean reset usually means restarting the Hardhat node, redeploying the contracts, restarting Alto, and restarting Next.js.

One additional Alto-specific issue can occur on a fresh local chain: the deterministic deployment sender used by the bundler may have no balance. On Hardhat, the account can be funded explicitly through JSON-RPC if needed:

```
cat >/tmp/alto-fund.json <<'JSON'
{"jsonrpc": "2.0", "id": 1, "method": "hardhat_setBalance",
 "params": ["0x3fAB184622Dc19b6109349B94811493BF2a45362",
 "0x8AC7230489E80000"]}
JSON
```

Symptom	Fix
<code>spawn precontract_cli</code> ENOENT Missing <code>deployed-contracts.</code> <code>json</code> or missing library addresses	Build the native binary from <code>src/wasm</code> with <code>cargo build -release -bin precontract_cli</code> . Start Hardhat and rerun <code>./deploy-all.sh</code> .
Alto says the EntryPoint does not exist	The Hardhat chain was probably restarted. Redeploy with <code>./deploy-all.sh</code> and then restart Alto.
Frontend reports Failed to fetch for UserOperations EACCES in <code>src/hardhat/artifacts</code>	Check that <code>./run-alto.sh</code> is running and that the bundler RPC is reachable at http://localhost:4337/rpc . Some files were created by another user. Restore ownership of <code>src/hardhat/artifacts</code> and <code>src/hardhat/cache</code> .
Native 1 GiB benchmark skips itself	Build both required native binaries: <code>precontract_cli</code> and <code>hardcoded_sha256_dispute_cli</code> .
Browser/WASM struggles on very large files	Use the native CLI benchmark path. The 1 GiB result in this report is a native CLI result, not a browser result.

Table 11.2: Common setup and execution issues.

```
curl -X POST http://127.0.0.1:8545 \
  -H "Content-Type: application/json" \
  --data @/tmp/alto-fund.json
```

After this, restart `run-alto.sh`. This command is only a local development fix; it is not part of the SOX protocol.

Chapter 12

Conclusion

12.1 Summary of the project

This semester project started from an existing implementation of the Sponsored Fair Exchange protocol and turned it into a more structured, measurable, and gas-aware research prototype. The goal was not to redesign the cryptographic protocol from first principles. The goal was to understand the inherited implementation, identify which protocol variants matter for gas consumption, implement the most relevant ones, and measure them under clearly defined scopes.

The first part of the work was therefore investigative. The existing system combined Rust and WebAssembly precontract generation, Solidity optimistic and dispute contracts, a Next.js/Electron interface, a SQLite backend, and an ERC-4337-style bundler path. Before changing the protocol logic, it was necessary to reconstruct how these components actually interacted. This included rebuilding the local stack, mapping UI and contract actions to the steps of the protocol, and identifying where deployment, sponsorship, dispute triggering, and proof generation really occurred in the code.

The second part of the work focused on variants. The project implemented and tested the main optimistic simplifications: no S -deposit, $S = B$, and $S = V$. It also implemented the relevant self-sponsored dispute behavior for $S_B = B$ and $S_V = V$, and a dedicated hardcoded SHA256 dispute mode for $desc = \text{SHA256}(x)$. These variants were first made functional in a monolithic implementation, then reorganized into a final specialized architecture with smaller contracts and clone-based optimistic deployments.

The third part of the work was experimental. The project produced same-scope gas measurements for the initial implementation, the monolithic intermediate implementation, and the final specialized implementation. It also measured off-chain runtime for large-file precontract and dispute witness generation. The most important large-file result is that a complete hardcoded SHA256 dispute was executed for a 1 GiB input using the native Rust pipeline.

12.2 Summary of results

The main positive result is on the optimistic path. In the initial implementation, an honest optimistic exchange cost 2,300,201 gas, including deployment and execution. In the final clone-based specialized architecture, the marginal optimistic cost per exchange falls to 500,720 gas for `no_S_deposit`, 516,349 gas for $S = B$, and 556,309 gas for $S = V$, once the reusable infrastructure has been deployed. This is the clearest gas reduction obtained in the project.

The second result is methodological but important: the intermediate monolithic implementation was not an optimization. It was a functional intermediate step. It made the variants executable, but it increased gas because the deployed bytecode contained many mutually exclusive branches. This negative result directly motivated the final split architecture. It also made clear that gas comparisons must distinguish one-time infrastructure costs, marginal per-exchange costs, dispute deployment, post-trigger dispute execution, and local Step 8 costs.

The third result is the dispute-side specialization. On the same 4 MiB dispute initialization scope, the final specialized implementation costs 5,495,184 gas, compared with 7,054,365 gas for the initial implementation and 8,872,096 gas for the monolithic intermediate implementation. On the 16 KiB same-scope dispute comparison, all four final specialized variants improve over their monolithic counterparts, with the self-sponsored hardcoded case reaching 4,042,185 gas in the measured scope.

The fourth result is the hardcoded SHA256 mode. In this mode, the selected circuit is fixed by the precontract metadata. The dispute contract reconstructs the relevant gate in Step 8 and no longer requires the generic circuit membership proof π_1 . Locally, this saves 62,542 gas on the measured 16 KiB Step 8 call and about 166,000–169,000 gas on the isolated 1 GiB $h_{circuit}$ check. In the full specialized implementation, the hardcoded path also reduces the dispute deployment component because it uses a dedicated deployer and dispute account.

The fifth result is practical scalability. The native Rust pipeline can handle the 1 GiB hardcoded benchmark. The measured self-sponsored hardcoded path executes the optimistic setup, the dispute trigger, 26 challenge rounds, Step 8, and finalization for a total of 13,990,092 gas. This is a concrete end-to-end result for the implemented benchmark path. It should be interpreted correctly: it is a native CLI benchmark, not a browser-only workflow.

12.3 Final assessment

The project achieved its main implementation and measurement objectives. The inherited system was made executable and analyzed. The main sponsoring variants were specified, implemented, and tested. The monolithic

approach was identified as gas-inefficient and replaced by a specialized architecture. The optimistic path was substantially reduced in marginal gas cost. The hardcoded `SHA256` dispute mode was implemented with the contract reconstructing the gate instead of relying on a vendor-provided generic gate and π_1 . Finally, the project demonstrated a 1 GiB dispute path using native tooling.

At the same time, the result remains a research prototype. The implementation still uses a local development UI, local accounts, a local backend, and Hardhat measurements. The optimized clone paths do not provide a full production wallet and ERC-4337 experience. The precontract verification route exists, but acceptance is not yet enforced by a backend verification precondition. The proof-generation API is also partial. These limitations are now documented explicitly rather than hidden behind the gas results.

The project also identified inherited cryptographic-format issues. The generic V2 circuit encoding is functional for the generated circuits used in the tests, but it is not the cleanest final specification because some gate arities are inferred rather than encoded explicitly. The Merkle-style accumulators also lack domain separation between leaves and internal nodes. These issues were not introduced by the new variants, but they should be fixed in a future protocol version before any production deployment or paper-ready specification.

Overall, the final implementation is stronger than the starting point in three ways. It is more gas-efficient on the main same-scope measurements, it is more explicit about which protocol variants are being executed, and it is more honest about its remaining security and engineering boundaries. This combination is important: a useful research prototype should not only produce lower numbers, but also make clear what those numbers mean.

12.4 Perspectives

The natural next step is not to add more flags to the existing contracts. The project showed that this path leads back to monolithic bytecode and confusing measurements. Future work should instead continue the specialization and deployment-architecture direction.

A first direction is a cleaner factory-first or registry-first design. Such an architecture could represent exchanges by identifiers or deterministic minimal proxies, reuse deployed logic more systematically, and make Step 1+2 fusion possible even when $S \neq B$. This would require a careful redesign of signatures, state layout, and ERC-4337 validation, but it is the most coherent path for reducing repeated deployment costs further.

A second direction is a new commitment format. A V3 circuit encoding should remove ambiguity by encoding arity explicitly or by using distinct opcodes for cases such as one-input versus two-input SHA-256 gates and

zero-input versus one-input CONST gates. The Merkle accumulators should also be domain-separated, for example by using different hash prefixes for leaves and internal nodes. This would make the implementation cleaner from a cryptographic engineering perspective.

A third direction is completing the verification workflow. In a production-quality system, the buyer should not be able to accept a precontract unless the relevant verification has succeeded and the verified values are bound to the exchange. Similarly, the dispute UI should make actor-side verification explicit before each signed action. This would turn the existing verification tools into enforced protocol workflow steps.

A fourth direction is optimizing the remaining dispute bottlenecks. After the optimistic path improvements, the dominant costs are dispute deployment, `triggerDispute`, and Step 8 evaluation, especially for AES-related gates and the associated h_{ct}/h_{pre} proofs. Further work should target these components directly.

Finally, the 1 GiB benchmark suggests a practical architecture for large-file fair exchange: the frontend should orchestrate the user workflow, while native backend workers generate large cryptographic artifacts. Turning this into a robust user-facing system would require streaming, progress reporting, artifact availability, wallet integration, and production deployment assumptions. The present project provides the measurements and implementation foundation needed to make that next step precise.

Appendix A

Detailed Gas Tables

This appendix collects the main gas tables used in Chapter 8. The goal is not to introduce new measurements, but to make the accounting recomposable in one place. All values are gas units measured from Hardhat transaction receipts unless explicitly stated otherwise.

A.1 Initial and monolithic optimistic paths

Component	Initial	Interm. mono.	Difference
Optimistic deployment	2,077,362	2,813,862	+736,500
Optimistic execution	222,839	232,920	+10,081
Total	2,300,201	3,046,782	+746,581

Table A.1: Same-scope optimistic-path comparison.

A.2 Factory and Clone Costs

One-time component	Gas
<code>S0XFactory</code>	561,126
Normal clone implementation	1,492,453
<code>no_S_deposit</code> implementation	1,482,093
$S = B$ implementation	1,529,267
$S = V$ implementation	1,519,062
Total one-time infrastructure	6,584,001

Table A.2: One-time clone infrastructure cost.

Mode	Create	Payment	Key	Complete	Total
<code>no_S_deposit</code>	299,694	82,272	61,363	57,391	500,720
$S = B$	393,412	0	61,363	61,574	516,349
$S = V$	328,926	104,446	61,363	61,574	556,309

Table A.3: Marginal optimistic costs once the clone infrastructure is deployed.

Specialized/factory quantity	Recomposition	Gas
One-time optimistic infrastructure	561,126 + 1,492,453 + 1,482,093 + 1,529,267 + 1,519,062	6,584,001
<code>no_S_deposit</code> marginal optimistic exchange	299,694 + 82,272 + 61,363 + 57,391	500,720
$S = B$ marginal optimistic exchange	393,412 + 0 + 61,363 + 61,574	516,349
$S = V$ marginal optimistic exchange	328,926 + 104,446 + 61,363 + 61,574	556,309
4 MiB final specialized dispute-init scope	348,946 + 5,146,238	5,495,184

Table A.4: Explicit recomposition of the final factory/clone specialized costs.

Table A.4 is the accounting view of the final factory-based specialized implementation. The first line is a one-time protocol setup cost. The following optimistic lines are marginal per-exchange costs after this setup. The last line gives the same-scope 4 MiB final specialized dispute-init total used for comparison with the initial and monolithic implementations.

A.3 Same-scope dispute comparisons

Measure, 4 MiB	Initial	Interm. mono.	Final specialized
Optimistic deployment / creation	2,077,362	2,813,862	348,946
Dispute deployment / trigger	4,651,201	5,779,296	4,862,324
Exchange + dispute triggering	4,977,003	6,058,222	5,146,238
Optimistic deploy + dispute init total	7,054,365	8,872,096	5,495,184
One challenge round average	82,624	103,282	103,128
Step 8 SHA-256 gate	320,000	379,686	382,181

Table A.5: Same-scope 4 MiB comparison across the three implementation stages.

16 KiB final specialized case	SB	Trigger	Respond	Opinion	Step 8	Total
Normal, external sponsors	118,094	4,862,324	729,630	763,872	880,050	7,432,647
Self- sponsored	109,415	4,593,958	364,815	381,986	420,072	5,946,445
Hardcoded SHA-256	115,588	2,719,333	729,630	763,872	790,225	5,195,022
Self- sponsored + hardcoded	106,954	2,721,171	364,815	381,936	393,435	4,042,185

Table A.6: Detailed 16 KiB final specialized dispute accounting. Finalization is included in the total.

A.4 Hardcoded strict end-to-end path

Final hardcoded strict component	16 KiB	1 GiB
Optimistic deployment	2,668,072	2,668,072
Buyer payment	104,441	104,441
Vendor key	58,896	58,896
$S_B = B$ dispute deposit	106,954	106,954
Total optimistic before trigger	2,938,363	2,938,363
Trigger / dispute deployment	2,721,171	2,721,171
Challenge responses total	608,190	1,581,282
Opinions total	421,615	1,095,903
Step 8 left submission	5,449,017	5,577,706
Finalization	75,667	75,667
Total complete path	12,214,023	13,990,092

Table A.7: Complete hardcoded strict path with $S_B = B$ and $S_V = V$.

A.5 Monolithic hardcoded mirror

Table A.8 gives the matching hardcoded breakdown for the intermediate monolithic path. This is not the final architecture. It corresponds to `OptimisticSOXAccount` with a separate hardcoded-configuration call and the monolithic dispute account. In this path, π_1 is absent, but the selected gate is still supplied as 64 bytes of `gateBytes`; the final specialized path removes both π_1 and the supplied gate description.

Monolithic hardcoded component	16 KiB	1 GiB
Optimistic deployment	2,813,682	2,813,682
Hardcoded configuration	81,897	81,897
Buyer payment	104,441	104,441
Vendor key	58,896	58,896
$S_B = B$ dispute deposit	106,954	106,954
Total optimistic before trigger	3,165,870	3,165,870
Trigger / dispute deployment	5,789,186	5,789,186
Total optimistic + trigger	8,955,056	8,955,056
Challenge responses total	608,850	1,582,998
Opinions total	422,495	1,098,191
Step 8 left submission with <code>gateBytes</code>	5,461,150	5,576,406
Finalization	75,667	75,667
Dispute after trigger	6,568,162	8,333,262
Total complete path	15,523,218	17,288,318

Table A.8: Complete monolithic hardcoded mirror with $S_B = B$ and $S_V = V$.

For the 1 GiB hardcoded path with external sponsors and three Step 7–8 iterations, the measured dispute after trigger is 23,865,946 gas. Including the trigger gives 26,585,279 gas, and including the optimistic path gives 29,532,277 gas. This value is not obtained by multiplying the one-iteration path by three, because later iterations update storage slots that have already been initialized.

Appendix B

Detailed Off-Chain Timing Tables

This appendix gathers the off-chain timings that are used in the discussion of large-file execution. They are wall-clock timings on the benchmark machine, not gas measurements. The values are therefore useful for relative interpretation and reproducibility, but they are not protocol constants.

B.1 Native 1 GiB benchmark timings

Measurement	Time	Interpretation
Precontract CLI	22.42–48.70 s	Vendor-side native preparation of ciphertext, commitments, circuit metadata, and opening values.
Native dispute generation	16.43–35.46 s	Aggregate native benchmark time for preparing dispute data; not attributable to one actor only.
Total $hpre_i$ generation	9.47–11.92 s	Buyer-side generation of challenge responses over 26 rounds in the forced-left benchmark.
Local Hardhat transactions	0.30–0.56 s	Local transaction submission time in the test environment; not a public-chain latency measurement.

Table B.1: Observed off-chain timing ranges for repeated 1 GiB hardcoded benchmark runs.

B.2 First forced-left $hpre_i$ rounds

Round	Challenge index i	Native $hpre_i$ time
0	16,777,219	4.58–6.04 s
1	8,388,610	2.23–2.46 s
2	4,194,305	1.17–1.33 s
3	2,097,153	0.60–0.71 s
4	1,048,577	0.31–0.39 s

Table B.2: Observed first $hpre_i$ timing ranges in repeated 1 GiB forced-left dispute benchmarks.

The decreasing times in Table B.2 are CPU times, not gas. The benchmark forces the left branch of the binary search, so the challenge index is approximately divided by two at each round. The native computation recomputes a smaller prefix for later rounds.

B.3 Native versus browser/WASM interpretation

Path	Role in the project
WebAssembly frontend path	Useful for integration, UI demonstrations, and small or medium files. It is not the reliable path for 1 GiB measurements.
Native Rust CLI path	Used for large-file precontract generation and hardcoded dispute witnesses. The 1 GiB results in this report are native CLI results.
Hardhat transaction time	Measures local execution of test transactions. It does not include public-network latency, mempool behavior, or wallet interaction delays.

Table B.3: Interpretation of the off-chain execution paths.

In the direct WASM experiments, files around 640 MiB and 704 MiB could be processed after memory-related patches, while larger sizes such as 736 MiB remained unreliable in that path. For this reason, the report claims 1 GiB support only for the native CLI benchmark path.

Appendix C

Protocol-Step Mapping Table

The implementation uses function names that do not exactly match the step numbering of the protocol paper. This appendix records the mapping used for all measurements.

C.1 Scope definitions

The most important caution is that `triggerDispute` should not be interpreted as a small state transition. In the measured implementation, it often includes deployment of a new dispute account, which explains why it dominates several complete dispute totals.

Implementation action	Protocol step	Measurement meaning
Contract creation / initialization	Step 1	Deployment or clone creation of the optimistic exchange account. In the clone architecture, this is a marginal proxy creation once the implementation exists.
sendPayment	Step 2	Buyer payment. In the $S = B$ optimized path this payment is fused with creation/initialization.
sendKey	Step 3	Vendor key release in the honest optimistic path.
Buyer-side sponsor fee call	Step 4–5	Buyer-side dispute sponsor registration and deposit. In the $S_B = B$ case this is logically self-sponsored.
Dispute trigger / vendor-side fee	Step 6	Vendor-side dispute sponsor registration and deployment of the dispute account. In many tests this is a large transaction because it includes deployment.
respondChallenge	Step 7b	Buyer publishes the current $hpre_i$ value for the binary search.
giveOpinion	Step 7c	Vendor accepts or rejects the buyer's response, updating the binary-search interval.
Commitment submission	Step 8	On-chain verification of the disputed gate, authenticated values, and accumulator proofs.
Dispute finalization	Step 9	Finalization or cancellation after the dispute outcome.

Table C.1: Mapping between implementation actions and protocol steps.

Scope	Included operations
Optimistic path	Step 1, Step 2, Step 3, and completion when no dispute occurs.
Dispute trigger	Step 6, often including vendor-side sponsor deposit and deployment of the dispute account.
Dispute after trigger	Step 7b, Step 7c, Step 8, and Step 9 after the dispute account exists.
Complete dispute path	Optimistic setup, trigger, challenge rounds, Step 8, and finalization.
Marginal clone cost	Per-exchange cost after factory and implementation contracts are already deployed.
Fixed infrastructure cost	One-time deployment of reusable factory or implementation contracts.

Table C.2: Gas-accounting scopes used throughout the report.

Appendix D

Contract and Script Reference

This appendix lists the main implementation files that are relevant for reproducing and auditing the project. It is a navigation aid, not a substitute for the code.

D.1 Solidity contracts

Component	Role
SOXFactory	Creates minimal optimistic clones for the precontract-time variants.
Optimistic clone contracts	Specialized optimistic escrow implementations for normal, no-deposit, $S = B$, and $S = V$ modes.
Direct normal optimistic account	Inherited direct normal account, including the ERC-4337-compatible path used by the original application.
Hardcoded optimistic account	Direct optimistic account for the hardcoded $desc = \text{SHA256}$ path.
Normal dispute account	Generic dispute account for normal disputes.
Self-sponsored dispute account	Specialized Step 9 handling for $S_B = B$ and $S_V = V$.
Hardcoded dispute account	Hardcoded dispute account that reconstructs SHA-256 gates without π_1 .
Hardcoded SHA-256 circuit library	Library reconstructing the expected hardcoded SHA-256 circuit gates from public metadata.

Table D.1: Main Solidity components.

D.2 Rust and application components

File or directory	Role
<code>src/wasm/src/bin/precontract_cli.rs</code>	Native precontract generation.
<code>src/wasm/src/bin/check_precontract_cli.rs</code>	Native verification of received precontract data.
<code>src/wasm/src/bin/compute_proofs_cli.rs</code>	Native helper for the current proof generation API.
<code>src/wasm/src/bin/hardcoded_sha256_dispute_cli.rs</code>	Native generation of hardcoded dispute witnesses for large-file benchmarks.
<code>src/app/api/precontracts/verify/route.ts</code>	Backend route invoking the native precontract checker.
<code>src/app/api/precontracts/accept/route.ts</code>	Proof-of-concept acceptance route; it does not enforce verification as a backend precondition.
<code>src/app/components/user/NewContractModal.tsx</code>	User-facing precontract creation and variant selection.
<code>src/app/lib/protocol-variants.ts</code>	TypeScript definition of user-facing protocol variants.

Table D.2: Main Rust, backend, and frontend components.

D.3 Scripts and benchmark files

Script or test	Role
<code>install.sh</code>	Installs dependencies, initializes the database, and prepares the bundler.
<code>deploy-all.sh</code>	Deploys the local EntryPoint, simulation contracts, SOX contracts, factory, and implementation addresses.
<code>run-alto.sh</code>	Starts the Alto bundler on the local chain.
<code>start-web.sh</code> / <code>npm run dev</code>	Starts the Next.js frontend.
Initial-scope comparison test	Remeasures the current repository in the same scope as the initial implementation.
Specialized architecture test	Measures the specialized contract architecture and bytecode-related costs.
Final same-scope test	Measures final specialized contracts in same-scope 4 MiB and 16 KiB scenarios.
Native 1 GiB hardcoded test	Executes the native hardcoded 1 GiB full-dispute benchmark.

Table D.3: Main scripts and benchmark entry points.

Appendix E

Demo Workflow

This appendix gives a concise script for reproducing the demonstration shown by the application. The recommended demonstration file size is 4 MiB, because it is large enough to demonstrate real off-chain processing while remaining practical for a screen recording.

E.1 Preparation

```
mkdir -p demo_files
truncate -s 4M demo_files/demo_4MiB.bin
```

Start the services in separate terminals:

```
cd src/hardhat && npx hardhat node
./deploy-all.sh
./run-alto.sh
npm run dev
```

Then open <http://localhost:3000>.

E.2 Optimistic exchange demonstration

1. Log in or select the vendor role in the local development UI.
2. Create a new precontract.
3. Select the buyer address.
4. Upload `demo_files/demo_4MiB.bin`.
5. Keep the normal sponsored mode for the standard full optimistic demo, or select SIMSOX, $S = B$, or $S = V$ when demonstrating the new optimistic variants.

6. Submit the precontract and wait for native precontract generation.
7. Switch to the buyer role.
8. Open the non-accepted precontract.
9. Click “Verify commitment” before accepting.
10. Accept the precontract; the ciphertext is downloaded by the buyer interface.
11. Switch to the sponsor role and sponsor/deploy the optimistic account.
12. Switch to the buyer role and send the payment.
13. Switch to the vendor role and send the key.
14. Verify that the optimistic contract reaches the complete state.

This demonstration covers the honest path. It is the clearest video workflow because it shows the real application, the precontract generation, the buyer verification button, the ciphertext transfer, contract deployment, payment, key release, and completion.

E.3 Dispute demonstration note

The dispute path is better demonstrated through the Hardhat benchmarks than through the current UI. The benchmark tests execute the challenge rounds, Step 8, and finalization in a controlled way with reproducible gas receipts. If a live dispute video is required, it should use a small file first, because preparing and submitting dispute witnesses manually is more fragile than the optimistic UI flow.

E.4 Recording notes

For a presentation video, the recommended structure is:

1. Briefly state that the demo uses a local Hardhat network and a 4 MiB file.
2. Show the vendor creating the precontract.
3. Show the buyer verifying and accepting the precontract.
4. Show the sponsor deploying the optimistic contract.
5. Show the buyer payment and vendor key release.

6. End on the complete state and mention that gas/dispute benchmarks are reproduced by the test suite rather than by hand in the UI.

This avoids overclaiming: the video demonstrates the application workflow, while the quantitative dispute claims remain tied to reproducible tests.

Appendix F

Additional Test Logs and Reproducibility Notes

F.1 Clean measurement discipline

The gas measurements in this report were interpreted under the following rules:

- never compare a fixed infrastructure cost with a marginal per-exchange cost;
- never compare a local Step 8 cost with a complete dispute total without saying so;
- restart or reset the Hardhat chain when deployment addresses become stale;
- rebuild native CLIs before running native large-file benchmarks;
- report whether a number comes from the initial, monolithic, or final specialized implementation.

These rules were introduced after early measurements mixed scopes and were difficult to explain. They are now part of the experimental methodology.

F.2 Canonical benchmark commands

```
cd src/hardhat
```

```
npx hardhat test \  
  test/performance/HanaComparisonCurrent.test.ts  
npx hardhat test \  
  test/performance/SpecializedContractArchitecture.test.ts
```

```
npx hardhat test \  
  test/performance/SpecializedFinalSameScope.test.ts
```

For the native hardcoded benchmark:

```
cd src/wasm  
cargo build --release --bin precontract_cli \  
  --bin hardcoded_sha256_dispute_cli  
  
cd ../hardhat  
npx hardhat test \  
  test/performance/NativeHardcodedSHA256FullDispute1GB.test.ts
```

For a small smoke test of the same code path:

```
SOX_FULL_DISPUTE_SIZE_BYTES=16384 npx hardhat test \  
  test/performance/NativeHardcodedSHA256FullDispute1GB.test.ts
```

F.3 Interpreting local benchmark output

Hardhat gas values come from transaction receipts and are stable for a fixed contract version, optimizer configuration, and test path. Wall-clock times are less stable because they depend on the host machine, file system, CPU load, and whether the benchmark is run inside WSL or directly on Linux.

The 1 GiB benchmark creates a temporary sparse file through truncation. This makes the test practical without storing a permanent 1 GiB fixture in the repository. The benchmark still executes the precontract CLI and dispute CLI on that file and then performs the on-chain dispute path.

F.4 Known local-environment issues

- If `run-alto.sh` reports that the `EntryPoint` does not exist, the Hardhat chain was probably restarted after deployment. Redeploy with `./deploy-all.sh`.
- If a native CLI is missing, rebuild it from `src/wasm` with `cargo build --release --bins`.
- If Hardhat cannot write under `src/hardhat/artifacts`, the files may have been created by another user. Restore ownership before recompiling.
- If the web application has stale exchanges, reset `src/app/db/sox.sqlite` and replay `src/app/db/init.sql`.

F.5 Submitted-code boundary

The repository contains historical code, intermediate monolithic contracts, final specialized contracts, scripts, and benchmark files. The report uses the following terminology consistently:

- initial implementation: the baseline inherited from the previous project;
- intermediate monolithic implementation: the variant-rich contracts before the final split;
- final specialized implementation: factory, clones, specialized disputes, and the hardcoded strict path.

This boundary is important for grading and future reuse. The project contribution is not only the final gas number; it is also the separation of these stages, the implementation of the retained variants, and the reproducible measurement framework.

Bibliography

- [1] Serge Vaudenay. *Sponsored Fair Exchange*. Cryptology ePrint Archive, Paper 2026/904, 2026. <https://eprint.iacr.org/2026/904>.
- [2] Hana El Moutaoukil. *Sponsored Fair Exchange: Implementation*. Semester project report, EPFL/LASEC, September 2025.